

Graphentheorie

Eine algorithmisch-orientierte Einführung
mit Graphprofilen, Signatur-Methode und Anwendungen

Stephan Epp

Universität Bielefeld

5. Juni 2026

Inhaltsverzeichnis

Vorwort	iv
1 Graphen – Grundbegriffe und Strukturen	1
1.1 Grundlegende Definitionen	1
1.1.1 Knotengrade	2
1.1.2 Wege und Kreise	3
1.1.3 Zusammenhang	3
1.2 Operationen mit Graphen	3
1.3 Spezielle Graphklassen	4
1.4 Isomorphie von Graphen	5
1.5 Aufgaben	5
2 Graphen und Matrizen	6
2.1 Die Adjazenzmatrix	6
2.1.1 Potenzen der Adjazenzmatrix und Boolean-Darstellung	7
2.1.2 Zerlegbare Matrizen und Zusammenhang	8
2.2 Inzidenzmatrix und Gradmatrix	8
2.3 Abstände in Graphen und das Kantenmaß	8
2.4 Spannbäume und die Laplace-Matrix	9
2.5 Aufgaben	10
3 Planare Graphen und die Eulersche Polyederformel	11
3.1 Planare Einbettungen	11
3.2 Die Eulersche Polyederformel	11
3.3 Nichtplanare Graphen und der Satz von Kuratowski	12
3.4 Der duale Graph	13
3.5 Aufgaben	13
4 Unabhängige Mengen, Cliques und Matchings	14
4.1 Unabhängige Knotenmengen	14
4.2 Matchings	15
4.3 Aufgaben	15
5 Färbungen von Graphen	16
5.1 Knotenfärbungen	16
5.2 Färbungen planarer Graphen	17
5.3 Das chromatische Polynom	17
5.4 Aufgaben	17

6	Der Zusammenhang von Graphen	18
6.1	Knotenzusammenhang	18
6.2	Kantenzusammenhang	18
6.3	Trade-off zwischen Kantenmaß und Durchmesser	19
6.4	Isoperimetrische Zahl und Robustheit	19
6.5	Aufgaben	19
7	Bäume	20
7.1	Charakterisierungen von Bäumen	20
7.2	Zählung von Bäumen	21
7.3	Wurzelbäume und Binärbäume	21
7.4	Aufgaben	22
8	Kreise in Graphen	23
8.1	Kreisstruktur und Basiskreise	23
8.2	Eulerkreise und Eulerwege	23
8.3	Hamiltonkreise	24
8.4	Aufgaben	24
9	Gerichtete Graphen	25
9.1	Grundlegende Eigenschaften	25
9.1.1	DAGs und topologische Sortierung	25
9.2	Starker Zusammenhang	26
9.3	Flüsse in gerichteten Graphen	27
9.4	Kortikale Informationsverarbeitung als Digraph	27
9.5	Aufgaben	27
10	Spezielle Gebiete der Graphentheorie	29
10.1	Extremale Graphentheorie	29
10.2	Graphen und Symmetrie	30
10.3	Chordale Graphen	30
10.4	Dominierende Mengen	31
10.5	Aufgaben	31
11	Graphenalgorithmen und Graphprofilverteilung	32
11.1	Das Graphprofil	32
11.2	Boolean Matrixmultiplikation mit Signaturen	32
11.3	Tiefensuche (DFS)	34
11.3.1	Brücken und Artikulationen via DFS	35
11.4	Breitensuche (BFS) und kürzeste Wege	35
11.5	Der Algorithmus von Dijkstra	36
11.6	Floyd-Warshall – alle paarweisen kürzesten Wege	37
11.7	Vollständige Profilberechnung	38
11.8	Graphprofile biologischer Netzwerke	39
11.9	Python-Implementierung	40
11.10	Aufgaben	40

12 Ausblick und offene Forschungsfragen	41
12.1 Parallelisierung der Profilberechnung	41
12.2 Sparse Graphen und komprimierte Darstellungen	41
12.3 Dynamische Graphen	42
12.4 Approximative Varianten	42
12.5 Quantenalgorithmen	42
12.6 Universelle Graphdatenbank	42
12.7 Neuronale Netzwerke und künstliche Intelligenz	43
12.8 Theoretische Komplexitätsfragen	43
12.9 Epistemische Netzwerke und kognitive Graphtheorie	43
12.10 Eine universelle Theorie der Graphprofilverteilung	43
Symbolverzeichnis	45
Literaturverzeichnis	46
Stichwortverzeichnis	48

Vorwort

Graphen gehören zu den mächtigsten und vielseitigsten Abstraktionen der Informatik und Mathematik. Ein Graph besteht aus einer Menge von Knoten und einer Menge von Kanten, die Paare von Knoten verbinden. Diese schlichte Struktur genügt, um eine außerordentlich breite Klasse realer Systeme zu modellieren: Straßennetze, Computernetzwerke, soziale Beziehungsgeflechte, Proteininteraktionen, logische Abhängigkeiten in Software sowie – besonders eindrucksvoll – die synaptischen Verbindungen zwischen Neuronen im menschlichen Gehirn.

Die Stärke des Graphmodells liegt in seiner Abstraktion. Sobald ein System als Graph formalisiert ist, lassen sich allgemeine algorithmische Werkzeuge anwenden, unabhängig vom jeweiligen Anwendungsgebiet. Fragen wie *Wie weit sind zwei Knoten voneinander entfernt?*, *Gibt es einen Weg von A nach B?* oder *Welche Knoten sind für die Konnektivität besonders kritisch?* übersetzen sich direkt in algorithmische Probleme auf Graphen.

Das Besondere dieser Einführung ist die konsequente *algorithmische Perspektive*. Neben der klassischen Graphentheorie – entwickelt entlang der Kapitelstruktur eines anerkannten Lehrbuches – wird die *Graphprofilverteilung* als zentrales Analysewerkzeug eingeführt. Das *Profil* eines Graphen – das Tripel aus kürzester-Wege-Matrix D , längster-Wege-Matrix L und Kantenmaß κ – erlaubt eine optimale, deterministische Charakterisierung jedes Graphen und bildet die Grundlage für den Vergleich von Netzwerken verschiedenster Domänen.

Ein weiteres Alleinstellungsmerkmal ist die *Signatur-Methode* für die Boolean Matrixmultiplikation. Diese Technik, bei der jede Zeile einer Boolean-Matrix als ganzzahlige Signatur kodiert wird, ermöglicht eine Laufzeit von $O(n^2)$ für die Matrixmultiplikation und damit $O(n^3)$ für die vollständige Profilberechnung – jeweils bei $O(n^2)$ Speicher. Alle Algorithmen werden mit vollständigen Korrektheits- und Laufzeitnachweisen präsentiert.

Aufbau des Buches. Kapitel 1 legt die Grundbegriffe (Graphen, Wege, Zusammenhang) fest. Kapitel 2 führt die algebraischen Darstellungen (Adjazenzmatrix, Laplace-Matrix, Kantenmaß) ein und beweist den Zusammenhang zwischen Matrixpotenzen und Wegen. Kapitel 3 behandelt planare Graphen und die Eulersche Polyederformel. Kapitel 4 untersucht unabhängige Mengen, Cliques und Matchings. Kapitel 5 widmet sich der Graphfärbung inklusive des Vier-Farben-Satzes und des chromatischen Polynoms. Kapitel 6 analysiert den Zusammenhang von Graphen quantitativ. Kapitel 7 behandelt Bäume, die Cayleysche Formel und den Prüfercode. Kapitel 8 untersucht Kreise: Eulerkreise, Hamiltonkreise und den Zyklusraum. Kapitel 9 führt gerichtete Graphen mit Anwendungen in Flüssen und neuronalen Netzwerken ein. Kapitel 10 behandelt spezielle Gebiete: extremale Graphentheorie, Symmetrie, chordale Graphen und dominierende Mengen. Kapitel 11 – das Kernkapitel – präsentiert die Graphenalgorithmen (DFS, BFS,

Dijkstra, Floyd-Warshall, vollständige Profilberechnung) mit sämtlichen Korrektheits- und Laufzeitnachweisen sowie die Graphprofilverteilung mit experimentellen Daten. Kapitel 12 gibt einen ausführlichen Ausblick auf offene Forschungsfragen.

Voraussetzungen. Das Buch setzt mathematische Grundkenntnisse voraus, wie sie in den ersten Studiensemestern der Informatik oder Mathematik vermittelt werden: elementare Mengenlehre, vollständige Induktion, Grundlagen der Linearen Algebra. Programmierkenntnisse in Python sind für die Quelltextbeispiele hilfreich, aber nicht zwingend erforderlich.

Bielefeld, 5. Juni 2026

Stephan Epp

Kapitel 1

Graphen – Grundbegriffe und Strukturen

Graphen gehören zu den mächtigsten und vielseitigsten Abstraktionen der Informatik und Mathematik. Ein Graph besteht aus einer Menge von Knoten und einer Menge von Kanten, die Paare von Knoten verbinden. Diese schlichte Struktur genügt, um eine außerordentlich breite Klasse realer Systeme zu modellieren: Straßennetze, Computernetzwerke, soziale Beziehungsgeflechte, Proteininteraktionen, logische Abhängigkeiten in Software sowie – besonders eindrucksvoll – die synaptischen Verbindungen zwischen Neuronen im menschlichen Gehirn.

Die Stärke des Graphmodells liegt in seiner Abstraktion. Sobald ein System als Graph formalisiert ist, lassen sich allgemeine algorithmische Werkzeuge anwenden, unabhängig vom jeweiligen Anwendungsgebiet. Fragen wie *Wie weit sind zwei Knoten voneinander entfernt?*, *Gibt es einen Weg von A nach B?* oder *Welche Knoten sind für die Konnektivität besonders kritisch?* übersetzen sich direkt in algorithmische Probleme auf Graphen, deren Lösung in allen Anwendungsdomänen gleichermaßen nützlich ist.

1.1 Grundlegende Definitionen

Definition 1.1 (Graph). Ein (*ungerichteter*) Graph $G = (V, E)$ besteht aus einer endlichen nichtleeren *Knotenmenge* V und einer *Kantenmenge* $E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$. Die Anzahl der Knoten $n = |V|$ heißt *Ordnung*, die Anzahl der Kanten $m = |E|$ heißt *Größe* des Graphen. Wir schreiben auch $V(G)$ für die Knotenmenge und $E(G)$ für die Kantenmenge von G .

Bemerkung 1.1. Die Bezeichnungen V und E entstammen den englischen Wörtern *vertex* (Knoten) und *edge* (Kante). In der gesamten Arbeit verwenden wir $n = |V|$ und $m = |E|$ als Standardnotation. Wenn wir von einem Graphen ohne weitere Zusatz sprechen, meinen wir stets einen endlichen, schlichten, ungerichteten Graphen.

Definition 1.2 (Schlinge, Parallele Kanten, Schlichter Graph, Multigraph). Eine Kante $\{v, v\}$, deren beide Endknoten zusammenfallen, heißt *Schlinge*. Zwei Kanten, die dieselben Endknoten verbinden, heißen *parallele Kanten*. Ein Graph ohne Schlingen und ohne parallele Kanten heißt *schlichter Graph*. Ein Graph, der parallele Kanten oder Schlingen enthalten kann, heißt *Multigraph*. In dieser Arbeit betrachten wir – sofern nicht anders angegeben – ausschließlich schlichte Graphen.

Definition 1.3 (Gerichteter Graph, Digraph). Ein *gerichteter Graph* (kurz *Digraph*) $G = (V, E)$ besitzt eine Kantenmenge $E \subseteq V \times V$, bei der jede Kante (u, v) eine feste Richtung von u (dem *Startknoten*) nach v (dem *Zielknoten*) trägt. Eine Kante (u, v) in einem Digraphen heißt auch *Pfeil* oder *gerichtete Kante*.

1.1.1 Knotengrade

Definition 1.4 (Grad, Minimalgrad, Maximalgrad). Der *Grad* $\deg_G(v)$ eines Knotens $v \in V$ ist die Anzahl der Kanten in E , die v als Endknoten besitzen:

$$\deg_G(v) = |\{e \in E \mid v \in e\}|.$$

Wir schreiben kurz $\deg(v)$, wenn der Graph G aus dem Kontext klar ist. Der *Minimalgrad* ist $\delta(G) = \min_{v \in V} \deg(v)$ und der *Maximalgrad* ist $\Delta(G) = \max_{v \in V} \deg(v)$. Ein Knoten mit $\deg(v) = 0$ heißt *isolierter Knoten*; ein Knoten mit $\deg(v) = 1$ heißt *Blatt*.

Definition 1.5 (Nachbarschaft). Die *offene Nachbarschaft* eines Knotens v ist die Menge seiner Nachbarknoten: $N(v) = \{u \in V \mid \{u, v\} \in E\}$. Die *geschlossene Nachbarschaft* ist $N[v] = N(v) \cup \{v\}$. Es gilt $\deg(v) = |N(v)|$.

Satz 1.1 (Handshaking-Lemma). Für jeden Graphen $G = (V, E)$ gilt:

$$\sum_{v \in V} \deg(v) = 2|E| = 2m.$$

Insbesondere ist die Anzahl der Knoten mit ungeradem Grad stets gerade.

Beweis. Jede Kante $\{u, v\} \in E$ hat genau zwei Endknoten. Daher trägt sie zur Summe $\sum_v \deg(v)$ genau zweimal bei – einmal für u und einmal für v . Somit gilt $\sum_v \deg(v) = 2m$. Da $2m$ eine gerade Zahl ist und die Summe aller Grade gerade ist, muss die Anzahl der Summanden, die ungerade sind (d. h. die Anzahl der Knoten mit ungeradem Grad), ebenfalls gerade sein. $\square$

Korollar 1.1. In jedem Graphen ist die Anzahl der Knoten mit ungeradem Grad gerade. Insbesondere hat jeder k -reguläre Graph (alle Grade gleich k) die Eigenschaft $k \cdot n = 2m$; eine solche Aussage lässt sich nicht mit k ungerade und n ungerade erfüllen.

Beispiel 1.1 (Gradfolgen). Wir betrachten einen Graphen G mit Knotenmenge $V = \{1, 2, 3, 4, 5\}$ und Kantenmenge $E = \{\{1, 2\}, \{1, 3\}, \{2, 3\}, \{3, 4\}, \{4, 5\}\}$. Dann ist $m = 5$ und die Grade sind: $\deg(1) = 2$, $\deg(2) = 2$, $\deg(3) = 3$, $\deg(4) = 2$, $\deg(5) = 1$. Die Summe $2 + 2 + 3 + 2 + 1 = 10 = 2 \cdot 5$ bestätigt das Handshaking-Lemma.

Definition 1.6 (Gradfolge, gradfolgenartig). Die *Gradfolge* eines Graphen ist die monoton nicht-wachsend sortierte Folge aller Knotengrade: $d_1 \geq d_2 \geq \dots \geq d_n$. Eine Folge nichtnegativer ganzer Zahlen $(d_1, \dots, d_n)$ mit $d_1 \geq \dots \geq d_n$ heißt *graphisch*, wenn es einen Graphen mit genau dieser Gradfolge gibt.

Satz 1.2 (Erdős–Gallai, 1960). Eine Folge $(d_1, d_2, \dots, d_n)$ mit $d_1 \geq d_2 \geq \dots \geq d_n \geq 0$ ist genau dann graphisch, wenn $\sum_{i=1}^n d_i$ gerade ist und für alle $k \in \{1, \dots, n\}$ gilt:

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

1.1.2 Wege und Kreise

Definition 1.7 (Weg, einfacher Weg, Länge). Ein *Weg* W der Länge k von Knoten u nach Knoten v ist eine endliche Folge $(v_0, v_1, \dots, v_k)$ von Knoten mit $v_0 = u$, $v_k = v$ und $\{v_l, v_{l+1}\} \in E$ für alle $0 \leq l < k$. Der Weg heißt *einfach*, wenn alle Knoten $v_0, v_1, \dots, v_k$ paarweise verschieden sind. Die *Länge* eines Weges ist die Anzahl seiner Kanten.

Definition 1.8 (Kreis, einfacher Kreis, Taille, Umfang). Ein *Kreis* ist ein geschlossener Weg $(v_0, v_1, \dots, v_k)$ mit $v_0 = v_k$ und $k \geq 1$. Ein Kreis heißt *einfach*, wenn alle Knoten $v_0, v_1, \dots, v_{k-1}$ paarweise verschieden sind. Die *Taille* $g(G)$ eines Graphen ist die Länge eines kürzesten Kreises (für kreisfreie Graphen: $g(G) = \infty$), der *Umfang* $\text{circ}(G)$ ist die Länge eines längsten Kreises. Ein Graph ohne Kreise heißt *azyklisch* oder *Wald*.

Bemerkung 1.2. Der Begriff *Weg* wird in der Literatur uneinheitlich verwendet. Manche Autoren (darunter Tittmann [1]) unterscheiden zwischen *Weg* (alle Kanten verschieden) und *einfachem Weg* (alle Knoten und Kanten verschieden). Wir verwenden im Folgenden *Weg* stets im Sinne eines einfachen Weges, sofern nicht anders angegeben.

1.1.3 Zusammenhang

Definition 1.9 (Zusammenhang, Zusammenhangskomponenten). Zwei Knoten $u, v \in V$ heißen *verbunden*, wenn es einen Weg von u nach v in G gibt. Ein Graph G heißt *zusammenhängend*, wenn je zwei seiner Knoten verbunden sind. Andernfalls heißt G *unzusammenhängend*. Die maximalen zusammenhängenden Teilgraphen von G heißen *Zusammenhangskomponenten* von G .

Definition 1.10 (Brücke, Artikulation). Eine Kante $e \in E$ heißt *Brücke*, wenn $G - e$ (der Graph mit entfernter Kante e) mehr Zusammenhangskomponenten als G hat. Ein Knoten $v \in V$ heißt *Artikulation*, wenn $G - v$ (der Graph mit entferntem Knoten v und allen inzidenten Kanten) mehr Zusammenhangskomponenten als G hat.

Satz 1.3. Eine Kante e ist genau dann eine Brücke, wenn sie in keinem Kreis von G enthalten ist.

Beweis. ($\Rightarrow$) Sei $e = \{u, v\}$ eine Brücke. Dann gilt: Wenn es einen Kreis C durch e gäbe, so gäbe es auch einen Weg von u nach v in $G - e$ (nämlich den Weg entlang C ohne die Kante e). Dies widerspricht der Tatsache, dass e eine Brücke ist. ($\Leftarrow$) Sei $e = \{u, v\}$ in keinem Kreis enthalten. Dann ist der einzige Weg von u nach v in G die Kante e selbst. Nach Entfernen von e sind u und v nicht mehr verbunden, also hat $G - e$ mehr Zusammenhangskomponenten als G . $\square$

1.2 Operationen mit Graphen

Definition 1.11 (Teilgraph, Induzierter Teilgraph, Spannender Teilgraph). Ein Graph $H = (V', E')$ heißt *Teilgraph* (Subgraph) von $G = (V, E)$, falls $V' \subseteq V$ und $E' \subseteq E$. H heißt *induzierter Teilgraph* von G durch V' , geschrieben $G[V']$, falls $E' = \{\{u, v\} \in E \mid u, v \in V'\}$, d. h. alle Kanten aus G zwischen Knoten von V' werden beibehalten. H heißt *spannender Teilgraph*, wenn $V' = V$.

Definition 1.12 (Komplement). Das *Komplement* $\overline{G}$ eines schlichten Graphen $G = (V, E)$ hat dieselbe Knotenmenge V und als Kantenmenge alle Paare, die in G fehlen: $E(\overline{G}) = \{\{u, v\} \mid u \neq v, \{u, v\} \notin E\}$. Es gilt $G \cup \overline{G} = K_n$ (vollständiger Graph).

Definition 1.13 (Kontraktion). Die *Kontraktion* G/e einer Kante $e = \{u, v\}$ entsteht, indem u und v zu einem neuen Knoten w verschmolzen werden und w mit allen früheren Nachbarn von u und v verbunden wird (außer u und v selbst).

1.3 Spezielle Graphklassen

Definition 1.14 (Vollständiger Graph K_n). Der *vollständige Graph* K_n hat n Knoten und enthält alle $\binom{n}{2} = \frac{n(n-1)}{2}$ möglichen Kanten. Jeder Knoten hat Grad $n - 1$. Das Kantenmaß (vgl. Kapitel 2) beträgt $\kappa(K_n) = \frac{2}{n-1} \rightarrow 0$ für $n \rightarrow \infty$.

Definition 1.15 (Pfadgraph P_n , Kreisgraph C_n). Der *Pfadgraph* P_n hat n Knoten $v_1, \dots, v_n$ und $n - 1$ Kanten $\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{n-1}, v_n\}$. Der *Kreisgraph* C_n entsteht aus P_n durch Hinzufügen der Kante $\{v_n, v_1\}$; er hat n Knoten und n Kanten. Das Kantenmaß beträgt $\kappa(P_n) = \frac{n}{n-1} \approx 1$ und $\kappa(C_n) = 1$.

Definition 1.16 (Baum, Wald). Ein *Baum* ist ein zusammenhängender, kreisfreier Graph. Ein Baum mit n Knoten hat genau $n - 1$ Kanten. Ein *Wald* ist ein kreisfreier (nicht notwendig zusammenhängender) Graph; seine Zusammenhangskomponenten sind Bäume.

Definition 1.17 (Bipartiter Graph, vollständiger bipartiter Graph). Ein Graph $G = (V, E)$ heißt *bipartit*, wenn die Knotenmenge V in zwei disjunkte Mengen A und B (die *Bipartitionsklassen*) zerlegt werden kann ($V = A \dot{\cup} B$), so dass jede Kante einen Endknoten in A und einen in B hat, d. h. $E \subseteq \{\{a, b\} \mid a \in A, b \in B\}$. Der *vollständige bipartite Graph* $K_{r,s}$ mit $|A| = r$ und $|B| = s$ enthält alle $r \cdot s$ möglichen Kanten zwischen A und B .

Satz 1.4 (Charakterisierung bipartiter Graphen). *Ein Graph G ist genau dann bipartit, wenn er keinen Kreis ungerader Länge enthält.*

Beweis. ($\Rightarrow$) Sei G bipartit mit Bipartitionsklassen A und B . Jeder Kreis wechselt mit jeder Kante zwischen A und B . Nach einer geraden Anzahl von Kanten ist man wieder in der Ausgangsklasse. Also hat jeder Kreis gerade Länge. ($\Leftarrow$) Sei G zusammenhängend (der allgemeine Fall folgt komponentenweise). Wähle einen beliebigen Startknoten s und färbe alle Knoten gerade Distanz von s mit Farbe A , alle mit ungerader Distanz mit Farbe B . Wenn eine Kante $\{u, v\}$ beide Endknoten in derselben Farbe hätte, so gäbe es einen Weg von s nach u , dann die Kante $\{u, v\}$, dann einen Weg von v nach s – das ergebe einen Kreis ungerader Länge, Widerspruch zur Voraussetzung. $\square$

Definition 1.18 (Regulärer Graph, Petersen-Graph). Ein Graph heißt *k -regulär*, wenn alle Knoten denselben Grad k besitzen. Bekannte Beispiele:

- Der *Petersen-Graph* ist 3-regulär mit 10 Knoten und 15 Kanten; er ist ein wichtiges Gegenbeispiel in der Graphentheorie.
- Der *Hyperkubus* Q_d ist d -regulär mit 2^d Knoten und $d \cdot 2^{d-1}$ Kanten; seine Knoten sind Binärwörter der Länge d , zwei Knoten sind verbunden genau dann, wenn sie sich in genau einer Stelle unterscheiden.

Definition 1.19 (Sterngraph S_n , Radgraph W_n). Der *Sterngraph* $S_n = K_{1,n}$ besteht aus einem Zentrums-knoten, der mit n Blattknoten verbunden ist; er hat $n + 1$ Knoten und n Kanten. Der *Radgraph* W_n entsteht aus C_n durch Hinzufügen eines Zentrums-knotens, der mit allen n Kreisknoten verbunden ist.

1.4 Isomorphie von Graphen

Definition 1.20 (Graphisomorphismus). Zwei Graphen $G_1 = (V_1, E_1)$ und $G_2 = (V_2, E_2)$ heißen *isomorph*, geschrieben $G_1 \cong G_2$, wenn es eine Bijektion $\phi : V_1 \rightarrow V_2$ gibt, so dass für alle $u, v \in V_1$ gilt:

$$\{u, v\} \in E_1 \Leftrightarrow \{\phi(u), \phi(v)\} \in E_2.$$

Die Abbildung ϕ heißt *Isomorphismus* von G_1 nach G_2 .

Lemma 1.1 (Notwendige Bedingungen für Isomorphie). *Sind $G_1 \cong G_2$, so gilt notwendigerweise:*

- $|V(G_1)| = |V(G_2)|$ (gleiche Ordnung),
- $|E(G_1)| = |E(G_2)|$ (gleiche Größe),
- Gleiche Gradfolge,
- Gleiche Anzahl von Zusammenhangskomponenten,
- Gleiche Taille und gleicher Umfang.

Diese Bedingungen sind jedoch nicht hinreichend für Isomorphie: Es gibt nicht-isomorphe Graphen, die dieselbe Gradfolge besitzen.

Beispiel 1.2 (Nicht-isomorphe Graphen mit gleicher Gradfolge). Die Graphen C_4 (Viereck) und $K_{1,3}$ plus isolierter Kante haben beide die Gradfolge $(2, 2, 2, 2)$ bzw. $(3, 1, 1, 1)$. Als ein subtileres Beispiel: Es gibt zwei nicht-isomorphe Graphen auf 6 Knoten mit Gradfolge $(3, 3, 2, 2, 2, 2)$ – nämlich den Kreis C_6 mit einer zusätzlichen Diagonale (zwei verschiedene Strukturen).

1.5 Aufgaben

Aufgabe 1.1. Zeigen Sie: In einem schlichten Graphen mit $n \geq 2$ Knoten gibt es stets zwei Knoten mit gleichem Grad. (*Hinweis:* Schubfachprinzip – die möglichen Grade liegen in $\{0, 1, \dots, n-1\}$, aber Grad 0 und Grad $n-1$ können nicht gleichzeitig auftreten.)

Aufgabe 1.2. Bestimmen Sie alle nicht-isomorphen schlichten Graphen mit 4 Knoten und 3 Kanten. (*Hinweis:* Es gibt genau zwei.)

Aufgabe 1.3. Berechnen Sie das Kantenmaß $\kappa = |V|/|E|$ für K_n , P_n , C_n und $K_{r,s}$. Welche Klasse hat das größte bzw. kleinste Kantenmaß asymptotisch?

Aufgabe 1.4. Zeigen Sie, dass für jeden k -regulären Graphen G mit n Knoten und k ungerade gilt: n ist gerade.

Aufgabe 1.5. Sei G ein Graph mit Gradfolge $(d_1, \dots, d_n)$. Zeigen Sie, dass der Komplementgraph $\overline{G}$ die Gradfolge $(n-1-d_1, \dots, n-1-d_n)$ hat.

Kapitel 2

Graphen und Matrizen

Die algebraische Behandlung von Graphen mittels Matrizen ist ein zentrales Werkzeug sowohl für theoretische Analysen als auch für algorithmische Berechnungen. Drei Matrizen sind besonders bedeutsam: die *Adjazenzmatrix*, die die Verbindungsstruktur kodiert; die *Inzidenzmatrix*, die die Beziehung zwischen Knoten und Kanten beschreibt; sowie die *Laplace-Matrix*, die spektrale Eigenschaften des Graphen kodiert und mit der Anzahl der Spannbäume zusammenhängt.

2.1 Die Adjazenzmatrix

Definition 2.1 (Adjazenzmatrix). Für einen Graphen $G = (V, E)$ mit n Knoten, deren Knoten wir mit $1, 2, \dots, n$ benennen, ist die *Adjazenzmatrix* $A \in \{0, 1\}^{n \times n}$ definiert durch:

$$A_{ij} = \begin{cases} 1 & \text{falls } \{i, j\} \in E, \\ 0 & \text{sonst.} \end{cases}$$

Für ungerichtete Graphen ist A symmetrisch ($A = A^\top$), da $\{i, j\} \in E \Leftrightarrow \{j, i\} \in E$.

Bemerkung 2.1. Die Adjazenzmatrix hängt von der Nummerierung der Knoten ab. Zwei Nummerierungen desselben Graphen liefern ähnliche Matrizen: Ist A_1 die Adjazenzmatrix bezüglich Nummerierung 1 und A_2 bezüglich Nummerierung 2, so gilt $A_2 = PA_1P^\top$ für eine geeignete Permutationsmatrix P . Deshalb sind zwei Graphen genau dann isomorph, wenn ihre Adjazenzmatrizen durch eine Permutationsmatrix ähnlich sind.

Beispiel 2.1 (Adjazenzmatrix des Pfadgraphen P_4). Für P_4 mit Knoten $\{1, 2, 3, 4\}$ und Kanten $\{\{1, 2\}, \{2, 3\}, \{3, 4\}\}$ ist:

$$A(P_4) = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Die Zeilen- und Spaltensummen ergeben die Knotengrade: $\deg(1) = 1, \deg(2) = 2, \deg(3) = 2, \deg(4) = 1$.

2.1.1 Potenzen der Adjazenzmatrix und Boolean-Darstellung

Das entscheidende Werkzeug für die spätere Profilberechnung ist die Verbindung zwischen Matrixpotenzen und Wegen im Graphen. Wir führen zunächst die Boolean Matrixmultiplikation ein, die anstelle der üblichen arithmetischen Operationen logische Operationen verwendet.

Definition 2.2 (Boolean Matrixmultiplikation). Für zwei Boolean-Matrizen $A, B \in \{0, 1\}^{n \times n}$ ist die *Boolean Matrixmultiplikation* definiert durch:

$$C_{ij} = \bigvee_{k=1}^n (A_{ik} \wedge B_{kj}),$$

wobei $\vee$ das logische ODER und $\wedge$ das logische UND bezeichnet. Äquivalent gilt: $C_{ij} = 1$ genau dann, wenn es ein k mit $A_{ik} = 1$ und $B_{kj} = 1$ gibt.

Lemma 2.1 (Wege und Matrixpotenzen). Sei A die Adjazenzmatrix eines Graphen $G = (V, E)$ und A^k die k -te Potenz unter Boolean Matrixmultiplikation. Dann gilt:

$$(A^k)_{ij} = 1 \Leftrightarrow \text{es existiert ein Weg der Länge } k \text{ von } i \text{ nach } j.$$

Beweis. Vollständige Induktion über $k \geq 1$.

Induktionsanfang ($k = 1$): Es gilt $(A^1)_{ij} = A_{ij} = 1$ genau dann, wenn $\{i, j\} \in E$, d. h. ein Weg der Länge 1 von i nach j existiert. ✓

Induktionsschritt ($k \rightarrow k + 1$): Sei die Aussage für k bewiesen. Dann gilt:

$$(A^{k+1})_{ij} = \bigvee_{l=1}^n ((A^k)_{il} \wedge A_{lj}).$$

Dieses Ergebnis ist genau dann $= 1$, wenn es ein l gibt mit $(A^k)_{il} = 1$ und $A_{lj} = 1$. Nach Induktionsvoraussetzung bedeutet $(A^k)_{il} = 1$, dass ein Weg der Länge k von i nach l existiert. Und $A_{lj} = 1$ bedeutet, dass die Kante $\{l, j\}$ existiert. Zusammen ergibt das einen Weg der Länge $k + 1$ von i nach j . Umgekehrt: Existiert ein Weg $(i = v_0, v_1, \dots, v_k, v_{k+1} = j)$ der Länge $k + 1$, so ist $(v_0, \dots, v_k)$ ein Weg der Länge k von i nach v_k (also $(A^k)_{i, v_k} = 1$ nach Induktionsvoraussetzung) und $\{v_k, j\} \in E$ (also $A_{v_k, j} = 1$). Damit ist $(A^{k+1})_{ij} \geq (A^k)_{i, v_k} \wedge A_{v_k, j} = 1$. □

Korollar 2.1. Für die Anzahl der Wege der Länge k zwischen i und j gilt bei der gewöhnlichen (nicht-Booleschen) Matrixmultiplikation: $(A^k)_{ij}$ ist gleich der Anzahl der (nicht notwendig einfachen) Wege der Länge k von i nach j .

Satz 2.1 (Erreichbarkeitsmatrix). Die Erreichbarkeitsmatrix $R \in \{0, 1\}^{n \times n}$ mit $R_{ij} = 1$ falls ein Weg von i nach j existiert (sonst 0) erfüllt:

$$R = A \vee A^2 \vee A^3 \vee \dots \vee A^{n-1},$$

wobei $\vee$ komponentenweise das logische ODER bezeichnet.

Beweis. Ein Weg von i nach j existiert genau dann, wenn ein Weg einer Länge $k \in \{1, \dots, n - 1\}$ von i nach j existiert. Die Schranke $n - 1$ ergibt sich, weil jeder einfache Weg in einem Graphen mit n Knoten höchstens $n - 1$ Kanten hat (mehr Kanten würde bedeuten, dass ein Knoten doppelt besucht wird). Nach Lemma 2.1 ist $(A^k)_{ij} = 1$ äquivalent zum Existieren eines Weges der Länge k . Die komponentenweise ODER-Verknüpfung liefert genau dann 1, wenn mindestens eine der Potenzen 1 ist. □

2.1.2 Zerlegbare Matrizen und Zusammenhang

Definition 2.3 (Zerlegbare und irreduzible Matrix). Eine quadratische Matrix $A \in \mathbb{R}^{n \times n}$ heißt *zerlegbar*, wenn es eine Permutationsmatrix P gibt, sodass $PAP^\top$ Blockdreiecksgestalt besitzt:

$$PAP^\top = \begin{pmatrix} B & C \\ 0 & D \end{pmatrix},$$

wobei B und D quadratische Matrizen sind. Andernfalls heißt A *irreduzibel*.

Satz 2.2. Die Adjazenzmatrix A eines Graphen G ist genau dann irreduzibel, wenn G zusammenhängend ist.

2.2 Inzidenzmatrix und Gradmatrix

Definition 2.4 (Inzidenzmatrix). Für einen Graphen $G = (V, E)$ mit n Knoten und m Kanten ist die *Inzidenzmatrix* $B \in \{0, 1\}^{n \times m}$ definiert durch:

$$B_{ie} = \begin{cases} 1 & \text{falls Knoten } i \text{ Endknoten der Kante } e \text{ ist,} \\ 0 & \text{sonst.} \end{cases}$$

Jede Spalte von B hat genau zwei Einsen (die beiden Endknoten der zugehörigen Kante). Es gilt $B\mathbf{1} = \deg$ (Gradvektor).

Definition 2.5 (Gradmatrix). Die *Gradmatrix* D_G ist die $n \times n$ -Diagonalmatrix mit $(D_G)_{ii} = \deg(i)$ und $(D_G)_{ij} = 0$ für $i \neq j$.

2.3 Abstände in Graphen und das Kantenmaß

Definition 2.6 (Abstand, Distanzmatrix). Der *Abstand* $d(u, v)$ zweier Knoten $u, v \in V$ ist die Länge eines kürzesten Weges von u nach v (bzw. ∞ , falls kein Weg existiert; $d(v, v) = 0$). Die *Distanzmatrix* $D \in (\mathbb{N}_0 \cup \{\infty\})^{n \times n}$ hat den Eintrag $D_{ij} = d(i, j)$.

Definition 2.7 (Durchmesser, Radius, Exzentrizität, Zentrum). Die *Exzentrizität* eines Knotens v ist $\varepsilon(v) = \max_{u \in V} d(u, v)$. Der *Durchmesser* ist $\text{diam}(G) = \max_{v \in V} \varepsilon(v) = \max_{u, v \in V} d(u, v)$. Der *Radius* ist $\text{rad}(G) = \min_{v \in V} \varepsilon(v)$. Das *Zentrum* ist die Menge der Knoten minimaler Exzentrizität: $Z(G) = \{v \in V \mid \varepsilon(v) = \text{rad}(G)\}$.

Satz 2.3. Für jeden zusammenhängenden Graphen G gilt: $\text{rad}(G) \leq \text{diam}(G) \leq 2 \text{rad}(G)$.

Beweis. Die erste Ungleichung $\text{rad}(G) \leq \text{diam}(G)$ folgt unmittelbar aus den Definitionen: Der Radius ist das Minimum, der Durchmesser das Maximum der Exzentrizitäten.

Für die zweite Ungleichung sei $c \in Z(G)$ ein Zentrumsknoten mit $\varepsilon(c) = \text{rad}(G)$. Für beliebige $u, v \in V$ gilt:

$$d(u, v) \leq d(u, c) + d(c, v) \leq \varepsilon(c) + \varepsilon(c) = 2 \text{rad}(G).$$

□

Definition 2.8 (Kantenmaß). Das *Kantenmaß* κ eines Graphen $G = (V, E)$ ist:

$$\kappa = \frac{|V|}{|E|} = \frac{n}{m}.$$

Ein kleines κ bedeutet viele Verbindungen pro Knoten (dichter Graph); ein großes κ kennzeichnet spärlich verbundene Topologien. Das Kantenmaß ist eine der drei Komponenten des Graphprofils (vgl. Kapitel 11).

Lemma 2.2 (Extremwerte des Kantenmaßes). *Für spezielle Graphklassen gilt:*

- Vollständiger Graph K_n : $\kappa(K_n) = \frac{2}{n-1} \xrightarrow{n \rightarrow \infty} 0$
- Pfadgraph P_n : $\kappa(P_n) = \frac{n}{n-1} \xrightarrow{n \rightarrow \infty} 1$
- Sterngraph $S_n = K_{1,n}$: $\kappa(S_n) = \frac{n+1}{n} \xrightarrow{n \rightarrow \infty} 1$
- Kreisgraph C_n : $\kappa(C_n) = 1$ für alle $n \geq 3$
- Vollständiger bipartiter Graph $K_{n,n}$: $\kappa(K_{n,n}) = \frac{2}{n} \xrightarrow{n \rightarrow \infty} 0$
- Graph ohne Kanten: $\kappa = \infty$

2.4 Spannbäume und die Laplace-Matrix

Definition 2.9 (Spannbaum). Ein *Spannbaum* eines zusammenhängenden Graphen G ist ein Teilgraph T , der alle Knoten von G enthält und ein Baum ist. Jeder zusammenhängende Graph besitzt mindestens einen Spannbaum.

Definition 2.10 (Laplace-Matrix). Die *Laplace-Matrix* (auch *Kirchhoff-Matrix*) eines Graphen G ist definiert als $L_G = D_G - A$, wobei D_G die Gradmatrix und A die Adjazenzmatrix ist. Es gilt:

$$(L_G)_{ij} = \begin{cases} \deg(i) & \text{falls } i = j, \\ -1 & \text{falls } \{i, j\} \in E, \\ 0 & \text{sonst.} \end{cases}$$

Die Laplace-Matrix erfüllt $L_G = BB^\top$ für die Inzidenzmatrix B (bei geeigneter Orientierung der Kanten).

Satz 2.4 (Kirchhoff, 1847 – Satz vom Matrix-Baum). *Sei G ein zusammenhängender Graph mit Laplace-Matrix L_G . Dann ist die Anzahl der Spannbäume von G gleich jedem Kofaktor von L_G , d. h. gleich dem (i, i) -Kofaktor $\det(L_G^{(i)})$, wobei $L_G^{(i)}$ die Matrix L_G ohne die i -te Zeile und i -te Spalte ist (für beliebiges i).*

Beispiel 2.2 (Anzahl der Spannbäume von K_4). Für den vollständigen Graphen K_4 beträgt die Laplace-Matrix:

$$L_{K_4} = \begin{pmatrix} 3 & -1 & -1 & -1 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ -1 & -1 & -1 & 3 \end{pmatrix}.$$

Der Kofaktor $\det(L_{K_4}^{(1)}) = 16 = 4^{4-2}$. Nach der Cayleyschen Formel hat K_n genau n^{n-2} Spannbäume; für $n = 4$ sind das $4^2 = 16$. ✓

Satz 2.5 (Eigenschaften der Laplace-Matrix). *Für jeden Graphen G gilt:*

- L_G ist symmetrisch und positiv semidefinit.
- Alle Eigenwerte von L_G sind reell und nicht-negativ: $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$.
- Die algebraische Vielfachheit des Eigenwerts 0 ist gleich der Anzahl der Zusammenhangskomponenten von G .
- Der kleinste positive Eigenwert λ_2 (der Fiedler-Wert) ist ein Maß für die Konnektivität des Graphen.

2.5 Aufgaben

Aufgabe 2.1. Berechnen Sie A^2 und A^3 für den Pfadgraphen P_4 und interpretieren Sie jeden Eintrag $(A^k)_{ij}$ graphentheoretisch.

Aufgabe 2.2. Bestimmen Sie Durchmesser, Radius und Zentrum für K_n , P_n , C_n und $K_{r,s}$.

Aufgabe 2.3. Berechnen Sie die Laplace-Matrix von P_4 und C_4 . Verifizieren Sie, dass die Spaltensummen null sind.

Aufgabe 2.4. Zeigen Sie: $\text{rad}(G) \leq \text{diam}(G) \leq 2 \text{rad}(G)$ und geben Sie für jede Seite ein Beispiel an, bei dem Gleichheit gilt.

Aufgabe 2.5. Bestimmen Sie die Erreichbarkeitsmatrix R des Graphen mit $V = \{1, 2, 3, 4\}$ und $E = \{\{1, 2\}, \{2, 3\}, \{3, 4\}\}$ durch sukzessive Berechnung von A, A^2, A^3 .

Kapitel 3

Planare Graphen und die Eulersche Polyederformel

Ein Graph heißt *planar*, wenn er in der Ebene so gezeichnet werden kann, dass sich keine zwei Kanten überschneiden. Diese Eigenschaft ist fundamental für Schaltkreisdesign, Kartographie und Computergrafik. Das wichtigste Resultat dieses Kapitels – die Eulersche Polyederformel – stellt eine elegante Beziehung zwischen Knoten, Kanten und Flächen her und erlaubt es, viele planare Graphen durch Kantenanzahlsschranken zu charakterisieren.

3.1 Planare Einbettungen

Definition 3.1 (Planarer Graph, planare Einbettung, Flächen). Ein Graph G heißt *planar*, wenn er eine *planare Einbettung* besitzt: eine Darstellung in der Ebene $\mathbb{R}^2$, bei der die Knoten als Punkte und die Kanten als Jordan-Kurven (stetige injektive Kurven) dargestellt werden, sodass sich zwei Kanten nur in gemeinsamen Endknoten schneiden.

Eine planare Einbettung zerlegt die Ebene in zusammenhängende offene Gebiete, die *Flächen* (Facetten) heißen. Genau eine dieser Flächen ist unbeschränkt; sie heißt *äußere Fläche*.

Bemerkung 3.1. Nicht jede geradlinige Zeichnung eines planaren Graphen ist planar, aber der Satz von Fary (1948) besagt, dass jeder planare Graph eine planare Einbettung besitzt, bei der alle Kanten geradlinige Segmente sind.

Definition 3.2 (Kreuzungszahl). Die *Kreuzungszahl* $cr(G)$ eines Graphen ist die minimale Anzahl von Kantenkreuzungen über alle möglichen Zeichnungen von G in der Ebene. Es gilt $cr(G) = 0$ genau dann, wenn G planar ist.

3.2 Die Eulersche Polyederformel

Satz 3.1 (Eulersche Polyederformel, Euler 1758). *Für jeden zusammenhängenden planaren Graphen G mit n Knoten, m Kanten und f Flächen gilt:*

$$n - m + f = 2.$$

Beweis. Wir beweisen die Formel durch Induktion über die Anzahl der Kanten m .

Induktionsanfang: Hat G einen einzigen Knoten und keine Kanten ($m = 0$), so gibt es $n = 1$ Knoten und $f = 1$ Fläche (die gesamte Ebene). Also $1 - 0 + 1 = 2$. ✓

Induktionsschritt: Sei $m \geq 1$ und die Formel für alle zusammenhängenden planaren Graphen mit weniger Kanten bewiesen.

- Falls G eine Brücke e enthält: Dann trennt $G - e$ in zwei Komponenten. Aber da G zusammenhängend war, hat $G - e$ genau zwei Zusammenhangskomponenten, die beide zusammenhängend sind. Tatsächlich hat $G - e$ weniger Kanten, ist aber nicht mehr zusammenhängend. Daher betrachten wir stattdessen den Fall:
- Falls G keinen Kreis enthält: Dann ist G ein Baum mit $m = n - 1$ Kanten und $f = 1$ Fläche. Dann $n - (n - 1) + 1 = 2$. ✓
- Falls G einen Kreis enthält: Sei e eine Kante dieses Kreises. Das Entfernen von e ergibt $G' = G - e$. G' ist immer noch zusammenhängend (da e in einem Kreis lag, also keine Brücke war). G' hat n Knoten, $m - 1$ Kanten und $f - 1$ Flächen (das Entfernen von e vereinigt zwei benachbarte Flächen zu einer). Nach Induktionsvoraussetzung: $n - (m - 1) + (f - 1) = 2$, also $n - m + f = 2$. ✓ □

Korollar 3.1 (Kantenschranken für planare Graphen). – Für jeden einfachen zusammenhängenden planaren Graphen mit $n \geq 3$ Knoten gilt: $m \leq 3n - 6$.

- Falls G zusätzlich bipartit ist ($n \geq 3$): $m \leq 2n - 4$.
- Aus $m \leq 3n - 6$ folgt: $\kappa = n/m \geq n/(3n - 6) \approx 1/3$ für planare Graphen.

Beweis. Jede Fläche ist von mindestens 3 Kanten begrenzt. Da jede Kante zu höchstens 2 Flächen gehört, gilt $3f \leq 2m$. Mit der Eulerschen Formel $f = 2 - n + m$ ergibt sich $3(2 - n + m) \leq 2m$, also $m \leq 3n - 6$. Für bipartite Graphen (Taille ≥ 4) gilt $4f \leq 2m$, was zu $m \leq 2n - 4$ führt. □

3.3 Nichtplanare Graphen und der Satz von Kuratowski

Satz 3.2 (K_5 und $K_{3,3}$ sind nicht planar). Die Graphen K_5 und $K_{3,3}$ sind nicht planar.

Beweis. K_5 : Hat $n = 5$, $m = 10$. Wäre K_5 planar, so müsste nach Korollar 3.1 gelten: $m \leq 3n - 6 = 9$. Da $10 > 9$, ist K_5 nicht planar.

$K_{3,3}$: Hat $n = 6$, $m = 9$. Da $K_{3,3}$ bipartit ist, müsste gelten: $m \leq 2n - 4 = 8$. Da $9 > 8$, ist $K_{3,3}$ nicht planar. □

Definition 3.3 (Unterteilung). Eine Unterteilung G' eines Graphen G entsteht, indem man Kanten von G durch Pfade ersetzt (d. h. man fügt auf einer Kante zusätzliche Knoten vom Grad 2 ein).

Satz 3.3 (Kuratowski, 1930). Ein Graph ist genau dann planar, wenn er keinen Teilgraphen enthält, der eine Unterteilung von K_5 oder $K_{3,3}$ ist.

Satz 3.4 (Wagner, 1937). Ein Graph ist genau dann planar, wenn er weder K_5 noch $K_{3,3}$ als Minor enthält (ein Graph H ist ein Minor von G , wenn H durch Kantenkontraktionen aus einem Teilgraphen von G entsteht).

3.4 Der duale Graph

Definition 3.4 (Dualer Graph). Sei G ein zusammenhängender planarer Graph mit einer planaren Einbettung. Der *duale Graph* G^* hat als Knotenmenge die Flächen von G ; für jede Kante $e \in E(G)$ gibt es eine Kante e^* in G^* , die die beiden Flächen verbindet, die e trennt (bei einer Brücke wird dieselbe Fläche zweifach verbunden, was eine Schlinge in G^* ergibt).

Satz 3.5. Für den dualen Graphen G^* eines zusammenhängenden planaren Graphen G gilt $(G^*)^* \cong G$.

3.5 Aufgaben

Aufgabe 3.1. Überprüfen Sie die Eulersche Formel für den Würfelgraphen Q_3 (8 Knoten, 12 Kanten) und bestimmen Sie die Anzahl seiner Flächen.

Aufgabe 3.2. Zeigen Sie, dass der Petersen-Graph nicht planar ist. (*Hinweis:* Zeigen Sie, dass er $K_{3,3}$ als Minor enthält.)

Aufgabe 3.3. Zeigen Sie: Jeder planare Graph enthält mindestens einen Knoten mit $\text{Grad} \leq 5$.

Kapitel 4

Unabhängige Mengen, Cliques und Matchings

Unabhängige Mengen und Matchings sind grundlegende Konzepte für Optimierungsprobleme in Graphen. Sie modellieren Konfliktsituationen (keine zwei ausgewählten Elemente dürfen benachbart sein) und Zuordnungsprobleme (maximale Paarungen zwischen Knoten). Anwendungen reichen von der Ressourcenzuteilung über Stundenplanerstellung bis hin zur DNA-Sequenzierung.

4.1 Unabhängige Knotenmengen

Definition 4.1 (Unabhängige Knotenmenge, Unabhängigkeitszahl). Eine Teilmenge $S \subseteq V$ heißt *unabhängige Knotenmenge* (oder *stabile Menge*), wenn keine zwei Knoten in S durch eine Kante verbunden sind: $\forall u, v \in S : \{u, v\} \notin E$. Die *Unabhängigkeitszahl* $\alpha(G)$ ist die maximale Größe einer unabhängigen Knotenmenge.

Definition 4.2 (Clique, Cliquenzahl). Eine Teilmenge $C \subseteq V$ heißt *Clique*, wenn je zwei Knoten in C durch eine Kante verbunden sind: C induziert einen vollständigen Teilgraphen $K_{|C|}$. Die *Cliquenzahl* $\omega(G)$ ist die maximale Größe einer Clique.

Satz 4.1 (Komplement-Beziehung). Für jeden Graphen G gilt $\alpha(G) = \omega(\overline{G})$ und $\omega(G) = \alpha(\overline{G})$.

Beweis. Eine unabhängige Menge in G ist genau eine Menge, bei der keine zwei Knoten eine Kante haben. Das bedeutet: alle Paare, die keine Kante in G bilden, bilden eine Kante in $\overline{G}$. Also ist eine unabhängige Menge in G genau eine Clique in $\overline{G}$. $\square$

Definition 4.3 (Knotenbedeckung, Bedeckungszahl). Eine Teilmenge $C \subseteq V$ heißt *Knotenbedeckung*, wenn jede Kante $\{u, v\} \in E$ mindestens einen Endknoten in C hat: $\forall \{u, v\} \in E : u \in C \text{ oder } v \in C$. Die *Bedeckungszahl* $\tau(G)$ ist die minimale Größe einer Knotenbedeckung.

Satz 4.2 (Gallai, 1959). Für jeden Graphen G ohne isolierte Knoten gilt: $\alpha(G) + \tau(G) = n = |V|$.

Beweis. Sei $S \subseteq V$ eine unabhängige Menge. Dann ist $V \setminus S$ eine Knotenbedeckung: Da S unabhängig ist, hat jede Kante $\{u, v\} \in E$ mindestens einen Endknoten in $V \setminus S$. Umgekehrt: Ist C eine Knotenbedeckung, so ist $V \setminus C$ unabhängig. Also ist S maximal unabhängig genau dann, wenn $V \setminus S$ minimal bedeckend ist. Damit gilt $\alpha(G) = n - \tau(G)$. $\square$

4.2 Matchings

Definition 4.4 (Matching, Sättigungsgrad, Matchingzahl). Eine Teilmenge $M \subseteq E$ heißt *Matching*, wenn keine zwei Kanten in M einen gemeinsamen Endknoten teilen. Ein Matching heißt *perfekt*, wenn jeder Knoten genau einem Endknoten einer Matchingkante ist (nur möglich bei geradem n). Die *Matchingzahl* $\nu(G)$ ist die Größe eines Matchings maximaler Größe (*maximales Matching*).

Definition 4.5 (Augmentierender Weg). Sei M ein Matching. Ein *augmentierender Weg* bezüglich M ist ein Weg, der an einem ungesättigten Knoten beginnt und endet und dessen Kanten abwechselnd nicht zu M und zu M gehören.

Satz 4.3 (Berge, 1957). *Ein Matching M ist genau dann ein Maximum-Matching (maximaler Größe), wenn kein augmentierender Weg bezüglich M existiert.*

Beweis. ($\Rightarrow$) Ist M ein Maximum-Matching und P ein augmentierender Weg, so kann man M durch die Kanten von P verbessern (symmetrische Differenz $M \Delta P$ ist ein größeres Matching) – Widerspruch.

($\Leftarrow$) Existiert kein augmentierender Weg, so sei M^* ein beliebiges Maximum-Matching. Die symmetrische Differenz $M \Delta M^*$ besteht aus Pfaden und Kreisen, bei denen Kanten von M und M^* abwechseln. Da $|M^*| > |M|$ angenommen, gibt es eine Komponente mit mehr Kanten aus M^* als aus M – das ist ein augmentierender Weg für M , Widerspruch. $\square$

Satz 4.4 (König, 1931). *In jedem bipartiten Graphen gilt: $\nu(G) = \tau(G)$. (Die Größe eines maximalen Matchings ist gleich der Größe einer minimalen Knotenbedeckung.)*

Satz 4.5 (Hall, 1935). *Sei $G = (A \cup B, E)$ ein bipartiter Graph. Dann besitzt G ein Matching, das alle Knoten von A sättigt, genau dann, wenn für alle $S \subseteq A$ gilt: $|N(S)| \geq |S|$ (Heiratsbedingung).*

4.3 Aufgaben

Aufgabe 4.1. Bestimmen Sie α , ω , ν und τ für den Petersen-Graphen.

Aufgabe 4.2. Zeigen Sie: In jedem k -regulären bipartiten Graphen existiert ein perfektes Matching.

Aufgabe 4.3. Geben Sie einen Graphen an, bei dem $\omega(G) < \chi(G)$. (*Hinweis: Berücksichtigen Sie den Fünf-Kreis C_5 .*)

Kapitel 5

Färbungen von Graphen

Graphfärbungen modellieren Konfliktsituationen, in denen benachbarte Elemente verschiedene Zustände einnehmen müssen: Kanaluweisung im Mobilfunk, Registerallokation in Compilern, Stundenplanerstellung, Kartographierung. Das zentrale Konzept ist die chromatische Zahl – die minimale Anzahl von Farben, die benötigt wird, um einen Graphen zulässig zu färben.

5.1 Knotenfärbungen

Definition 5.1 (Zulässige Knotenfärbung, chromatische Zahl). Eine (*zulässige*) k -Knotenfärbung von G ist eine Funktion $c : V \rightarrow \{1, 2, \dots, k\}$ (die k Farben), sodass je zwei benachbarte Knoten verschiedene Farben erhalten: $\forall \{u, v\} \in E : c(u) \neq c(v)$. Die *chromatische Zahl* $\chi(G)$ ist das minimale k , für das eine zulässige k -Färbung von G existiert.

Bemerkung 5.1. Offensichtlich gilt: $\chi(G) = 1$ genau dann, wenn G keine Kanten hat. $\chi(G) = 2$ genau dann, wenn G mindestens eine Kante und bipartit ist. $\chi(K_n) = n$ und $\chi(C_{2k}) = 2$, $\chi(C_{2k+1}) = 3$.

Satz 5.1 (Schranken für die chromatische Zahl). *Für jeden Graphen G gilt:*

$$\omega(G) \leq \chi(G) \leq \Delta(G) + 1.$$

Beweis. Untere Schranke: Jede Clique der Größe ω benötigt ω verschiedene Farben.

Obere Schranke (gieriger Algorithmus): Ordne die Knoten in beliebiger Reihenfolge. Weise jedem Knoten die kleinste Farbe zu, die nicht von einem bereits gefärbten Nachbarn verwendet wird. Da jeder Knoten höchstens $\Delta(G)$ Nachbarn hat, wird höchstens die $(\Delta(G) + 1)$ -te Farbe benötigt. $\square$

Satz 5.2 (Brooks, 1941). *Für jeden zusammenhängenden Graphen G , der weder ein vollständiger Graph noch ein ungerader Kreis ist, gilt: $\chi(G) \leq \Delta(G)$.*

Definition 5.2 (Kantefärbung, chromatischer Index). Eine k -Kantenfärbung weist jeder Kante eine Farbe aus $\{1, \dots, k\}$ zu, sodass keine zwei inzidenten Kanten dieselbe Farbe haben. Der *chromatische Index* $\chi'(G)$ ist das minimale k .

Satz 5.3 (Vizing, 1964). *Für jeden schlichten Graphen gilt: $\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1$.*

5.2 Färbungen planarer Graphen

Satz 5.4 (Fünf-Farben-Satz). *Jeder planare Graph ist 5-färbbar.*

Beweisidee. Induktion über n . Jeder planare Graph enthält einen Knoten v mit Grad ≤ 5 (folgt aus $m \leq 3n - 6$). Nach Entfernen von v ist der resultierende Graph nach Induktionsvoraussetzung 5-färbbar. Falls $\deg(v) \leq 4$, wird eine freie Farbe für v gefunden. Im Fall $\deg(v) = 5$ mit allen 5 Nachbarn in verschiedenen Farben verwendet man den Kempe-Ketten-Argument, um die Färbung umzustrukturieren. $\square$

Satz 5.5 (Vier-Farben-Satz, Appel & Haken, 1976). *Jeder planare Graph ist 4-färbbar: $\chi(G) \leq 4$.*

Bemerkung 5.2. Der Vier-Farben-Satz war das erste große mathematische Theorem, das wesentlich mit Hilfe eines Computers bewiesen wurde. Appel und Haken reduzierten das Problem auf die Überprüfung von 1936 Konfigurationen mit einem Computer. Ein vereinfachter Beweis wurde 1997 von Robertson, Sanders, Seymour und Thomas gegeben.

5.3 Das chromatische Polynom

Definition 5.3 (Chromatisches Polynom). Das *chromatische Polynom* $P(G, k)$ zählt die Anzahl der zulässigen k -Färbungen von G für $k \in \mathbb{N}$. Es ist ein Polynom in k vom Grad $n = |V|$ mit ganzzahligen Koeffizienten und führendem Term k^n .

Beispiel 5.1 (Chromatische Polynome). – Leerer Graph ($m = 0$): $P(G, k) = k^n$

- Vollständiger Graph: $P(K_n, k) = k(k-1)(k-2) \cdots (k-n+1)$
- Baum mit n Knoten: $P(T, k) = k(k-1)^{n-1}$
- Kreis C_n : $P(C_n, k) = (k-1)^n + (-1)^n(k-1)$

Satz 5.6 (Dekompositionsgleichung). *Für jede Kante $e = \{u, v\} \in E$:*

$$P(G, k) = P(G - e, k) + P(G/e, k),$$

wobei G/e den durch Kontraktion von e entstehenden Graphen bezeichnet.

Beweis. Die zulässigen k -Färbungen von $G - e$ zerfallen in zwei Klassen: solche, bei denen $c(u) \neq c(v)$ (diese sind auch zulässig in G), und solche, bei denen $c(u) = c(v)$ (diese entsprechen zulässigen Färbungen von G/e , bei dem u und v zu einem Knoten verschmolzen sind). Daher: $P(G - e, k) = P(G, k) + P(G/e, k)$. $\square$

5.4 Aufgaben

Aufgabe 5.1. Berechnen Sie $P(K_4, k)$ und $P(C_5, k)$ mit der Dekompositionsgleichung.

Aufgabe 5.2. Zeigen Sie: G ist genau dann bipartit, wenn $P(G, k)$ für alle geraden $k \geq 2$ einen positiven Wert annimmt.

Aufgabe 5.3. Bestimmen Sie $\chi(G)$ für den Petersen-Graphen.

Kapitel 6

Der Zusammenhang von Graphen

Der Zusammenhang eines Graphen quantifiziert, wie robust ein Netzwerk gegen Ausfälle ist. In Kommunikationsnetzen, Rechenzentren und biologischen Netzwerken ist es wichtig zu wissen, wie viele Knoten oder Kanten ausfallen können, bevor das Netz in mehrere nicht kommunizierende Teile zerfällt.

6.1 Knotenzusammenhang

Definition 6.1 (Knotenzusammenhangszahl, κ -zusammenhängend). Eine Menge $S \subseteq V$ heißt *trennende Knotenmenge*, wenn $G - S$ unzusammenhängend ist oder aus einem einzigen Knoten besteht. Die *Knotenzusammenhangszahl* $\kappa_0(G)$ ist die minimale Größe einer trennenden Knotenmenge. Ein Graph heißt *k-zusammenhängend*, wenn $\kappa_0(G) \geq k$.

Satz 6.1 (Whitney, 1932). Für jeden Graphen G gilt:

$$\kappa_0(G) \leq \kappa_1(G) \leq \delta(G),$$

wobei $\kappa_1(G)$ die *Kantenzusammenhangszahl* (s. u.) und $\delta(G)$ der *Minimalgrad* ist.

Beweis. $\kappa_1(G) \leq \delta(G)$: Sei v ein Knoten minimalen Grades $\delta(G)$. Das Entfernen aller $\delta(G)$ Kanten inzident zu v trennt v vom Rest des Graphen. Also $\kappa_1(G) \leq \delta(G)$.

$\kappa_0(G) \leq \kappa_1(G)$: Sei F eine minimale trennende Kantenmenge (Größe $\kappa_1(G)$). Für jede Kante $\{u, v\} \in F$ wählen wir einen der Endknoten (z. B. u). Die so gewählte Knotenmenge S trennt G , also $\kappa_0(G) \leq |S| \leq |F| = \kappa_1(G)$. $\square$

6.2 Kantenzusammenhang

Definition 6.2 (Kantenzusammenhangszahl, Schnitt). Eine Menge $F \subseteq E$ heißt *trennende Kantenmenge* (*Schnitt*), wenn $G - F$ unzusammenhängend ist. Die *Kantenzusammenhangszahl* $\kappa_1(G)$ ist die minimale Größe einer trennenden Kantenmenge. Ein Schnitt $(S, V \setminus S)$ für eine nichtleere echte Teilmenge $S \subset V$ besteht aus allen Kanten zwischen S und $V \setminus S$.

Satz 6.2 (Menger, 1927). Die *Kantenzusammenhangszahl* $\kappa_1(G)$ ist gleich der maximalen Anzahl kantendisjunkter Wege zwischen je zwei Knoten s und t . Analog ist $\kappa_0(G)$ gleich der maximalen Anzahl knotendisjunkter s - t -Wege.

6.3 Trade-off zwischen Kantenmaß und Durchmesser

Satz 6.3 (Trade-off: Kantenmaß und Durchmesser). *Für die in Kapitel 11 untersuchten biologisch inspirierten Graphklassen (je 20 Instanzen mit $n = 100$ Knoten) gilt empirisch:*

$$\text{diam}(G) \approx 4,2 + 12,8 \cdot \sqrt{\kappa}, \quad R^2 \approx 0,73.$$

Es gilt näherungsweise $\kappa \cdot \text{diam}(G) \approx 3,5$. Eine Halbierung des Durchmessers erfordert eine Verdopplung der Kantenzahl (Halbierung von κ).

Heuristischer Beweis. In einem sparse Graphen mit mittlerem Grad $\bar{d} = 2m/n = 2/\kappa$ ist die Anzahl der Knoten in Distanz $\leq k$ von einem Startknoten beschränkt durch $\bar{d}^k$. Der Durchmesser D erfüllt dann näherungsweise: $\bar{d}^D \approx n$, also $D \approx \log n / \log \bar{d} = \log n / \log(2/\kappa)$. Für moderate κ gilt approximativ $D \propto \sqrt{\kappa}$. $\square$

6.4 Isoperimetrische Zahl und Robustheit

Definition 6.3 (Isoperimetrische Zahl). Die *isoperimetrische Zahl* (auch *Cheeger-Konstante*) ist:

$$h(G) = \min_{\substack{S \subseteq V \\ 0 < |S| \leq n/2}} \frac{|\partial S|}{|S|},$$

wobei ∂S die Menge der Kanten zwischen S und $V \setminus S$ bezeichnet. Ein großes $h(G)$ bedeutet, dass jede Teilmenge viele Kanten nach außen hat – der Graph ist gut verbunden.

Satz 6.4. *Für den Fiedler-Wert λ_2 der Laplace-Matrix gilt: $\lambda_2/2 \leq h(G) \leq \sqrt{2\Delta\lambda_2}$.*

6.5 Aufgaben

Aufgabe 6.1. Berechnen Sie κ_0 und κ_1 für den Petersen-Graphen.

Aufgabe 6.2. Zeigen Sie: Für k -reguläre Graphen gilt $\kappa_1(G) \leq k$. Wann gilt Gleichheit?

Aufgabe 6.3. Bestimmen Sie die isoperimetrische Zahl für P_n , C_n und K_n .

Kapitel 7

Bäume

Bäume sind die einfachsten zusammenhängenden kreisfreien Graphen. Sie bilden das Gerüst vieler wichtiger Datenstrukturen in der Informatik (Suchbäume, Heaps, Syntaxbäume, Entscheidungsbäume) sowie hierarchischer Systeme in Natur und Technik (Stammbäume, Verzeichnisse, phylogenetische Bäume). Ihre Theorie ist vollständig und enthält schöne Sätze wie die Cayleysche Formel.

7.1 Charakterisierungen von Bäumen

Satz 7.1 (Äquivalente Charakterisierungen von Bäumen). *Für einen Graphen $G = (V, E)$ mit n Knoten sind folgende Aussagen äquivalent:*

1. *G ist ein Baum (zusammenhängend und kreisfrei).*
2. *G ist zusammenhängend und hat genau $n - 1$ Kanten.*
3. *G ist kreisfrei und hat genau $n - 1$ Kanten.*
4. *Je zwei Knoten sind durch genau einen einfachen Weg verbunden.*
5. *G ist kreisfrei, aber das Hinzufügen einer beliebigen neuen Kante $\{u, v\}$ mit $u, v \in V$, $\{u, v\} \notin E$ erzeugt genau einen Kreis.*
6. *G ist zusammenhängend, aber das Entfernen einer beliebigen Kante macht ihn unzusammenhängend (alle Kanten sind Brücken).*

Beweis. Wir zeigen $(1) \Rightarrow (2) \Rightarrow (3) \Rightarrow (4) \Rightarrow (1)$ und $(1) \Leftrightarrow (5) \Leftrightarrow (6)$.

$(1) \Rightarrow (2)$: Induktion über n . Für $n = 1$: $m = 0 = n - 1$. Für $n > 1$: Da jeder Baum mit $n \geq 2$ Knoten ein Blatt v hat (Grad 1), ist $G - v$ wieder ein Baum mit $n - 1$ Knoten und nach Induktionsannahme $n - 2$ Kanten. Also hat G genau $(n - 2) + 1 = n - 1$ Kanten.

$(2) \Rightarrow (3)$: Sei G zusammenhängend mit $n - 1$ Kanten. Angenommen G enthält einen Kreis C . Entfernen einer Kante aus C erhält den Zusammenhang, liefert also einen zusammenhängenden Graphen mit $n - 2$ Kanten. Dieser muss nach (2) einen Baum beinhalten, der $n - 1$ Kanten hat – Widerspruch.

$(3) \Rightarrow (4)$: Kreisfreiheit garantiert Eindeutigkeit der Wege; $n - 1$ Kanten sichern den Zusammenhang (ein kreisfreier Wald mit c Komponenten und n Knoten hat $n - c$ Kanten; für $m = n - 1$ folgt $c = 1$).

$(4) \Rightarrow (1)$: Eindeutige Wege implizieren Zusammenhang und Kreisfreiheit. □

Lemma 7.1 (Blätter im Baum). *Jeder Baum mit $n \geq 2$ Knoten hat mindestens zwei Blätter.*

Beweis. Die Summe aller Grade beträgt $2(n-1) = 2n-2$. Angenommen, es gibt höchstens ein Blatt (Grad 1). Dann haben $n-1$ Knoten Grad ≥ 2 . Die Summe der Grade ist also $\geq 1 + 2(n-1) = 2n-1 > 2n-2$ – Widerspruch. $\square$

7.2 Zählung von Bäumen

Satz 7.2 (Cayley, 1889). *Die Anzahl der markierten Bäume auf n Knoten beträgt n^{n-2} .*

Definition 7.1 (Prüfercode). Der *Prüfercode* kodiert einen markierten Baum auf $\{1, \dots, n\}$ als eine Folge $(a_1, a_2, \dots, a_{n-2}) \in \{1, \dots, n\}^{n-2}$. **Kodierung:** Wiederhole $n-2$ mal: Entferne das kleinste Blatt und notiere seinen einzigen Nachbarn. **Dekodierung:** Aus der Folge $(a_1, \dots, a_{n-2})$ kann der Baum eindeutig rekonstruiert werden.

Satz 7.3. *Der Prüfercode ist eine Bijektion zwischen markierten Bäumen auf $\{1, \dots, n\}$ und Folgen in $\{1, \dots, n\}^{n-2}$. Da es n^{n-2} solcher Folgen gibt, folgt der Satz von Cayley.*

Beispiel 7.1 (Prüfercode). Baum mit $n = 5$, Kanten $\{1, 3\}, \{2, 3\}, \{3, 4\}, \{4, 5\}$. Blätter: $\{1, 2, 5\}$. Schritt 1: Kleinstes Blatt = 1, Nachbar = 3. Code bisher: (3). Verbleibende Knoten: $\{2, 3, 4, 5\}$, Blätter: $\{2, 5\}$. Schritt 2: Kleinstes Blatt = 2, Nachbar = 3. Code: (3, 3). Verbleibende Knoten: $\{3, 4, 5\}$, Blätter: $\{5\}$ (und 3 nach Entfernen). Doch der Algorithmus endet hier (2 Schritte für $n-2 = 3$... nein, $n = 5$, also $n-2 = 3$ Schritte). Schritt 3: Kleinstes Blatt = 3, Nachbar = 4. Code: (3, 3, 4). Die letzte Kante ist $\{4, 5\}$. $\checkmark$

7.3 Wurzelbäume und Binärbäume

Definition 7.2 (Wurzelbaum, Höhe, Tiefe). Ein *Wurzelbaum* ist ein Baum mit einem ausgezeichneten Knoten r (*Wurzel*). Jeder Knoten $v \neq r$ hat einen eindeutig bestimmten *Elternknoten* $\text{pa}(v)$ (den Nachbarn von v auf dem eindeutigen Weg von v zur Wurzel). Die *Kinder* eines Knotens sind seine Nachbarn außer dem Elternknoten. Die *Tiefe* eines Knotens ist seine Distanz zur Wurzel. Die *Höhe* des Baumes ist die maximale Tiefe.

Definition 7.3 (Binärer Baum, vollständiger Binärbaum). Ein *binärer Baum* ist ein Wurzelbaum, in dem jeder innere Knoten höchstens zwei Kinder hat (linkes und rechtes Kind). Ein *vollständiger binärer Baum* der Höhe h hat genau $2^{h+1} - 1$ Knoten und 2^h Blätter.

Lemma 7.2 (Höhe und Blattanzahl). *Ein binärer Baum mit n Blättern hat Höhe $\geq \lceil \log_2 n \rceil$. Diese Schranke ist scharf (vollständiger Binärbaum).*

7.4 Aufgaben

Aufgabe 7.1. Bestimmen Sie alle nicht-isomorphen Bäume mit 6 Knoten.

Aufgabe 7.2. Berechnen Sie den Prüfercode des Sternbaums S_5 und des Pfades P_5 .
Rekonstruieren Sie anschließend den Baum aus dem Code.

Aufgabe 7.3. Zeigen Sie: Jeder Baum mit mindestens 2 Knoten ist 2-färbbar (bipartit).

Kapitel 8

Kreise in Graphen

Kreise sind fundamentale Strukturen in Graphen. Das Euler-Problem (Durchlauf aller Kanten genau einmal) ist eines der ältesten Probleme der Graphentheorie und wurde von Euler 1736 beim Königsberger Brückenproblem gelöst. Das Hamilton-Problem (Besuch aller Knoten genau einmal) hingegen ist NP-vollständig und bis heute theoretisch unvollständig verstanden.

8.1 Kreisstruktur und Basiskreise

Definition 8.1 (Zyklusraum). Der *Zyklusraum* $\mathcal{C}(G)$ eines Graphen G ist ein $\mathbb{F}_2$ -Vektorraum über den Kantenmengen aller Kreise. Die symmetrische Differenz zweier Kreise ist wieder eine Vereinigung von Kreisen. Die *Zyklendimension* ist $\dim \mathcal{C}(G) = m - n + c$, wobei c die Anzahl der Zusammenhangskomponenten ist.

Definition 8.2 (Kreisbasis, Fundamentalkreise). Zu einem aufspannenden Baum T von G und jeder Nicht-Baumkante $e \in E \setminus E(T)$ gibt es genau einen Kreis im Teilgraphen $T + e$; er heißt *Fundamentalkreis* bezüglich T . Die $m - n + 1$ Fundamentalkreise bilden eine *Kreisbasis* des Zyklusraums.

8.2 Eulerkreise und Eulerwege

Satz 8.1 (Euler, 1736). *Ein zusammenhängender Graph G besitzt einen Eulerkreis (geschlossene Kantentour, die jede Kante genau einmal durchläuft) genau dann, wenn alle Knoten geraden Grad haben.*

Beweis. ($\Rightarrow$) Sei C ein Eulerkreis. Jedes Mal, wenn der Kreis einen Knoten v besucht, nutzt er eine eingehende und eine ausgehende Kante. Daher trägt jeder Besuch $+2$ zum Grad von v bei. Also ist $\deg(v)$ gerade.

($\Leftarrow$) Alle Grade gerade. Konstruktion: Beginne an einem beliebigen Knoten v . Da alle Grade gerade sind, kann der Weg nie in einer Sackgasse enden (außer beim Startknoten), also gelangt man zu v zurück – ein Kreis C . Falls C noch nicht alle Kanten enthält, gibt es (da G zusammenhängend) einen Knoten $u \in C$ mit einer unbenutzten Kante. Starte einen neuen Kreis C' ab u ; dieser kann in C integriert werden. Das Verfahren terminiert mit einem Eulerkreis. $\square$

Korollar 8.1. *Ein zusammenhängender Graph besitzt einen Eulerweg (offene Tour durch jede Kante genau einmal) genau dann, wenn er genau zwei Knoten mit ungeradem Grad hat.*

Beispiel 8.1 (Königsberger Brückenproblem). Die Stadt Königsberg hatte 7 Brücken über den Pregel. Euler zeigte 1736, dass kein Weg existiert, der jede Brücke genau einmal überquert, da alle vier Landmassen einen ungeraden Knotengrad haben.

8.3 Hamiltonkreise

Definition 8.3 (Hamiltonkreis, -weg). Ein *Hamiltonkreis* ist ein Kreis, der jeden Knoten von G genau einmal besucht. Ein *Hamiltonweg* ist ein Weg, der jeden Knoten genau einmal besucht. Ein Graph heißt *hamiltonsch*, wenn er einen Hamiltonkreis besitzt.

Satz 8.2 (Dirac, 1952). *Sei G ein Graph mit $n \geq 3$ Knoten und $\delta(G) \geq n/2$. Dann ist G hamiltonsch.*

Beweis. Angenommen, G ist nicht hamiltonsch. Sei $P = (v_1, \dots, v_k)$ ein längster einfacher Weg in G . Da P maximal, sind alle Nachbarn von v_1 und v_k im Weg. Da $\deg(v_1) \geq n/2$, hat v_1 mindestens $n/2$ Nachbarn $v_{i_1+1}, \dots$ auf P . Ähnlich hat v_k mindestens $n/2$ Nachbarn $v_{j_1}, \dots$ auf P . Da $n/2 + n/2 = n > k - 1$, gibt es per Schubfachprinzip ein i , sodass v_1 zu v_{i+1} und v_k zu v_i adjazent ist. Damit existiert ein Hamilton-Kreis – Widerspruch. $\square$

Satz 8.3 (Ore, 1960). *Sei G mit $n \geq 3$ Knoten. Falls für je zwei nicht-adjazente Knoten u, v gilt: $\deg(u) + \deg(v) \geq n$, dann ist G hamiltonsch.*

Bemerkung 8.1 (NP-Vollständigkeit). Das Entscheidungsproblem, ob ein gegebener Graph einen Hamiltonkreis besitzt, ist NP-vollständig (Karp, 1972). Im Gegensatz dazu ist das Euler-Problem in $O(m)$ lösbar und das kürzeste-Wege-Problem in $O(n^3)$ (vgl. Kapitel 11).

8.4 Aufgaben

Aufgabe 8.1. Entscheiden Sie, ob K_5 , $K_{3,3}$, C_6 und der Petersen-Graph Eulerkreise, Eulerwege, Hamiltonkreise oder Hamiltonwege besitzen.

Aufgabe 8.2. Zeigen Sie: Jeder zusammenhängende reguläre Graph mit geradem Grad besitzt einen Eulerkreis.

Aufgabe 8.3. Geben Sie einen Graphen an, der Dirac-Bedingung erfüllt, und konstruieren Sie explizit einen Hamiltonkreis.

Kapitel 9

Gerichtete Graphen

Gerichtete Graphen (Digraphen) modellieren asymmetrische Beziehungen, bei denen die Richtung einer Verbindung entscheidend ist: Einbahnstraßen, Weblinks, Softwareabhängigkeiten, Informationsfluss in neuronalen Netzwerken. Viele der für ungerichtete Graphen entwickelten Konzepte lassen sich auf Digraphen übertragen, erfordern aber eine sorgfältige Anpassung.

9.1 Grundlegende Eigenschaften

Definition 9.1 (Digraph, In- und Außengrad). Ein *Digraph* $G = (V, E)$ hat eine Kantenmenge $E \subseteq V \times V$. Eine Kante $(u, v) \in E$ verläuft von u (*Quelle*) nach v (*Ziel*). Der *Innengrad* $\deg^-(v) = |\{u : (u, v) \in E\}|$ und der *Außengrad* $\deg^+(v) = |\{w : (v, w) \in E\}|$ erfüllen:

$$\sum_{v \in V} \deg^+(v) = \sum_{v \in V} \deg^-(v) = |E|.$$

Knoten mit $\deg^-(v) = 0$ heißen *Quellen*, solche mit $\deg^+(v) = 0$ heißen *Senken*.

Definition 9.2 (Gerichteter Weg, gerichteter Kreis). Ein *gerichteter Weg* der Länge k von u nach v ist eine Folge $(v_0, \dots, v_k)$ mit $v_0 = u$, $v_k = v$ und $(v_l, v_{l+1}) \in E$. Ein *gerichteter Kreis* ist ein geschlossener gerichteter Weg. Ein Digraph ohne gerichtete Kreise heißt *DAG* (directed acyclic graph, gerichteter azyklischer Graph).

9.1.1 DAGs und topologische Sortierung

Definition 9.3 (Topologische Sortierung). Eine *topologische Sortierung* eines DAGs ist eine lineare Ordnung der Knoten, sodass für jede Kante (u, v) der Knoten u vor v erscheint.

Satz 9.1. *Jeder DAG besitzt mindestens eine topologische Sortierung, die in $O(n + m)$ berechnet werden kann.*

Beweis. Jeder DAG hat mindestens eine Quelle (Knoten mit Innengrad 0, denn sonst gäbe es einen gerichteten Kreis). Entferne iterativ Quellen: Weise der Quelle die nächste Position zu, entferne sie, aktualisiere die Innengrade. Dies terminiert nach n Schritten. $\square$

Beispiel 9.1 (Abhängigkeitsgraph in Softwareprojekten). Softwaremodule bilden oft einen DAG, bei dem eine Kante (A, B) bedeutet: Modul A hängt von Modul B ab. Die topologische Sortierung gibt die Compilerreihenfolge an: Jedes Modul wird erst kompiliert, nachdem alle seine Abhängigkeiten fertig sind.

9.2 Starker Zusammenhang

Definition 9.4 (Schwacher und starker Zusammenhang, SCC). Ein Digraph heißt *schwach zusammenhängend*, wenn der zugrundeliegende ungerichtete Graph zusammenhängend ist. Er heißt *stark zusammenhängend*, wenn für je zwei Knoten u, v sowohl ein gerichteter Weg von u nach v als auch von v nach u existiert. Die maximalen stark zusammenhängenden Teilgraphen heißen *starke Zusammenhangskomponenten* (SCC, *strongly connected components*).

Satz 9.2 (Kondensation). Der Kondensationsgraph G^{SCC} eines Digraphen G – bei dem jede SCC zu einem Knoten verschmolzen wird – ist stets ein DAG. Er beschreibt die Makrostruktur des Graphen.

Algorithmus 9.1 Tarjans SCC-Algorithmus

Eingabe: Digraph $G = (V, E)$

Ausgabe: Menge aller starken Zusammenhangskomponenten

```

1: index  $\leftarrow 0$ ; Stack  $\leftarrow []$ ; SCCs  $\leftarrow []$ 
2: fuer alle  $v \in V$  tue
3:   falls  $v$  noch nicht besucht dann
4:     StarkVerbunden( $v$ )
5:   Ende falls
6: Ende fuer
7: procedure STARKVERBUNDEN( $v$ )
8:   disc[ $v$ ]  $\leftarrow$  low[ $v$ ]  $\leftarrow$  index++
9:   Lege  $v$  auf Stack; aufStack[ $v$ ]  $\leftarrow$  wahr
10:  fuer alle  $(v, w) \in E$  tue
11:    falls  $w$  nicht besucht dann
12:      StarkVerbunden( $w$ )
13:      low[ $v$ ]  $\leftarrow$  min(low[ $v$ ], low[ $w$ ])
14:    sonst falls aufStack[ $w$ ] dann
15:      low[ $v$ ]  $\leftarrow$  min(low[ $v$ ], disc[ $w$ ])
16:    Ende falls
17:  Ende fuer
18:  falls low[ $v$ ] = disc[ $v$ ] dann ▷  $v$  ist Wurzel einer SCC
19:    Entnehme Knoten vom Stack bis  $v$ : neue SCC
20:  Ende falls
21: Ende procedure
```

Satz 9.3 (Korrektheit und Laufzeit von Tarjan). Algorithmus 9.1 berechnet alle SCCs korrekt in Zeit $O(n + m)$.

Beweis. Korrektheit: Der Low-Link-Wert $\text{low}[v]$ ist die kleinste Entdeckungszeit, die von v aus über Baumkanten und höchstens eine Rückwärtskante erreichbar ist. Ein Knoten v ist Wurzel seiner SCC genau dann, wenn $\text{low}[v] = \text{disc}[v]$, d. h. kein Vorfahre von v ist über eine Rückwärtskante erreichbar.

Laufzeit: Jeder Knoten und jede Kante wird genau einmal verarbeitet: $O(n+m)$. $\square$

9.3 Flüsse in gerichteten Graphen

Definition 9.5 (Flussnetzwerk, zulässiger Fluss). Ein *Flussnetzwerk* ist ein Digraph $G = (V, E)$ mit Kapazitätsfunktion $c : E \rightarrow \mathbb{R}_{\geq 0}$, einer ausgezeichneten Quelle s und Senke t . Ein Fluss $f : E \rightarrow \mathbb{R}_{\geq 0}$ heißt *zulässig*, wenn gilt:

- *Kapazitätsbedingung:* $0 \leq f(e) \leq c(e)$ für alle $e \in E$.
- *Flusserhaltung:* Für alle $v \neq s, t$ gilt $\sum_{(u,v) \in E} f(u, v) = \sum_{(v,w) \in E} f(v, w)$.

Der *Flusswert* ist $|f| = \sum_{(s,v) \in E} f(s, v) - \sum_{(v,s) \in E} f(v, s)$.

Satz 9.4 (Max-Flow-Min-Cut, Ford & Fulkerson 1956). *In jedem Flussnetzwerk gilt: Maximaler Flusswert = minimale Kapazität eines s - t -Schnittes.*

Beweisidee. Es ist klar, dass kein Fluss die Kapazität eines Schnittes überschreiten kann (obere Schranke). Der Ford-Fulkerson-Algorithmus findet iterativ augmentierende Wege im Restgraph. Wenn kein augmentierender Weg mehr existiert, bilden die von s erreichbaren Knoten eine Seite eines Schnittes, dessen Kapazität genau dem aktuellen Flusswert entspricht. $\square$

9.4 Kortikale Informationsverarbeitung als Digraph

Beispiel 9.2 (Rechtsdrehung kortikaler Aktivierungswellen). Das menschliche Gehirn aus $86 \cdot 10^9$ Neuronen und ca. $1,5 \cdot 10^{14}$ Synapsen ist ein gewichteter Digraph. Von dorsal betrachtet propagiert die kortikale Aktivierungswelle im Uhrzeigersinn:

Frontal $\rightarrow$ Temporal-rechts $\rightarrow$ Parietal-rechts $\rightarrow$ Okzipital
 $\rightarrow$ Parietal-links $\rightarrow$ Temporal-links $\rightarrow$ Frontal.

Formal entspricht dies einer negativen Winkelgeschwindigkeit des kortikalen Aktivierungsvektors $\vec{A}(t)$:

$$\omega(t) = \frac{A_x(t) \dot{A}_y(t) - A_y(t) \dot{A}_x(t)}{|\vec{A}(t)|^2} < 0.$$

Das gesunde Gehirn ist damit ein *chiraler, stark zusammenhängender* Digraph mit strukturell bevorzugter Traversierungsrichtung. Ein vollständiger kortikaler Zyklus dauert bei ≈ 40 Hz genau 25 ms – ein *Gamma-Fenster*.

9.5 Aufgaben

Aufgabe 9.1. Bestimmen Sie alle starken Zusammenhangskomponenten des Digraphen mit $V = \{1, 2, 3, 4, 5\}$ und $E = \{(1, 2), (2, 3), (3, 1), (3, 4), (4, 5), (5, 4)\}$.

Aufgabe 9.2. Geben Sie eine topologische Sortierung für den DAG mit $V = \{A, B, C, D, E\}$ und $E = \{(A, C), (B, C), (B, D), (C, E), (D, E)\}$ an.

Aufgabe 9.3. Berechnen Sie den maximalen Fluss im Netz mit $V = \{s, a, b, c, t\}$ und Kapazitäten $s \rightarrow a:3$, $s \rightarrow b:2$, $a \rightarrow c:2$, $b \rightarrow c:3$, $a \rightarrow t:1$, $c \rightarrow t:4$.

Kapitel 10

Spezielle Gebiete der Graphentheorie

Dieses Kapitel beleuchtet ausgewählte Gebiete, die über die klassischen Grundlagen hinausgehen: extremale Graphentheorie, Graphen und Symmetrie, chordale Graphen sowie dominierende Mengen. Diese Bereiche sind sowohl theoretisch interessant als auch praktisch anwendbar – von der Netzwerkoptimierung bis zur Datenbanktheorie.

10.1 Extremale Graphentheorie

Die extremale Graphentheorie fragt, wie viele Kanten ein Graph haben kann, ohne einen bestimmten Teilgraphen zu enthalten. Turán und Ramsey sind die zentralen Sätze.

Satz 10.1 (Mantel, 1907). *Ein dreiecksfreier Graph auf n Knoten hat höchstens $\lfloor n^2/4 \rfloor$ Kanten. Gleichheit gilt für $K_{\lfloor n/2 \rfloor, \lceil n/2 \rceil}$.*

Beweis. Sei G dreiecksfrei. Für jede Kante $\{u, v\}$ gilt: $N(u)$ und $N(v)$ sind disjunkt (sonst gäbe es ein Dreieck). Also $\deg(u) + \deg(v) \leq n$. Summiert über alle Kanten: $\sum_{\{u,v\} \in E} (\deg(u) + \deg(v)) \leq mn$. Da die linke Seite $\sum_v \deg(v)^2 \geq (\sum_v \deg(v))^2/n = (2m)^2/n$ erfüllt, folgt $(2m)^2/n \leq mn$, also $m \leq n^2/4$. $\square$

Definition 10.1 (Turán-Graph $T(n, r)$). Der *Turán-Graph* $T(n, r)$ ist der vollständige r -partite Graph auf n Knoten mit möglichst gleich großen Klassen (Größen $\lfloor n/r \rfloor$ oder $\lceil n/r \rceil$). Er enthält kein K_{r+1} und maximiert die Kantenzahl unter allen K_{r+1} -freien Graphen.

Satz 10.2 (Turán, 1941). *Unter allen K_{r+1} -freien Graphen auf n Knoten hat $T(n, r)$ die meisten Kanten. Die Kantenzahl beträgt:*

$$\text{ex}(n, K_{r+1}) = \left(1 - \frac{1}{r}\right) \frac{n^2}{2} + O(n).$$

Definition 10.2 (Ramsey-Zahl). Die *Ramsey-Zahl* $R(s, t)$ ist die kleinste Zahl n , sodass jede 2-Färbung der Kanten von K_n (rot/blau) entweder einen roten K_s oder einen blauen K_t enthält.

Satz 10.3 (Ramsey, 1930). *Für alle $s, t \geq 2$ gilt $R(s, t) \leq \binom{s+t-2}{s-1}$. Insbesondere ist $R(3, 3) = 6$.*

Beweis für $R(3, 3) = 6$. Obere Schranke $R(3, 3) \leq 6$: In K_6 hat jeder Knoten 5 Nachbarn. Nach dem Schubfachprinzip sind mindestens 3 davon gleich gefärbt (blau oder rot). Falls diese 3 Nachbarn eine gleichfarbige Kante untereinander haben, gibt es ein einfarbiges Dreieck. Falls nicht, bilden sie ein anderfarbiges Dreieck. *Untere Schranke $R(3, 3) > 5$:* Färbe K_5 als Pentagon (rot) plus Pentagramm (blau) – kein einfarbiges Dreieck. $\square$

10.2 Graphen und Symmetrie

Definition 10.3 (Automorphismus, Automorphismengruppe). Ein *Automorphismus* von G ist ein Isomorphismus $\phi : G \rightarrow G$. Die Menge aller Automorphismen bildet die *Automorphismengruppe* $\text{Aut}(G)$ unter Komposition.

Beispiel 10.1 (Automorphismengruppen). – $\text{Aut}(K_n) \cong S_n$ (symmetrische Gruppe, $n!$ Elemente).

- $\text{Aut}(P_n) \cong \mathbb{Z}_2$ (Spiegelung am Mittelpunkt).
- $\text{Aut}(C_n) \cong D_n$ (Diedergruppe, $2n$ Elemente).
- $\text{Aut}(\text{Petersen}) \cong S_5$ (120 Elemente).

Definition 10.4 (Knotentransitiver und kantentransitiver Graph). G heißt *knoten-transitiv*, wenn $\text{Aut}(G)$ auf V transitiv wirkt: für je zwei Knoten u, v existiert ein Automorphismus mit $\phi(u) = v$. Knotentransitive Graphen sind regulär. G heißt *kantentransitiv*, wenn $\text{Aut}(G)$ auf E transitiv wirkt.

Satz 10.4 (Cayley-Graphen). Sei Γ eine endliche Gruppe und $S \subseteq \Gamma$ eine erzeugende Menge mit $S = S^{-1}$, $e \notin S$. Der Cayley-Graph $\text{Cay}(\Gamma, S)$ hat Knotenmenge Γ und Kantenmenge $\{\{g, gs\} \mid g \in \Gamma, s \in S\}$. Er ist $|S|$ -regulär und knotentransitiv.

10.3 Chordale Graphen

Definition 10.5 (Chordaler Graph, perfekte Eliminationsordnung). Ein Graph G heißt *chordal*, wenn jeder Kreis der Länge ≥ 4 eine *Sehne* (chord) besitzt – eine Kante zwischen zwei nicht aufeinanderfolgenden Kreisknoten. Äquivalent: G besitzt eine *perfekte Eliminationsordnung (PEO)* – eine lineare Ordnung der Knoten $v_1, \dots, v_n$, sodass für jeden Knoten v_i seine Nachbarn in $\{v_{i+1}, \dots, v_n\}$ eine Clique bilden.

Satz 10.5. Chordale Graphen haben polynomiell lösbare Probleme: $\chi(G) = \omega(G)$ (keine Differenz zwischen Clique- und chromatischer Zahl), und Maximum-Clique sowie maximale unabhängige Menge können in $O(n + m)$ berechnet werden.

Beispiel 10.2 (Intervallgraphen sind chordal). Ein *Intervallgraph* entsteht, wenn die Knoten reelle Intervalle repräsentieren und zwei Knoten genau dann verbunden sind, wenn die zugehörigen Intervalle sich überschneiden. Intervallgraphen sind stets chordal.

10.4 Dominierende Mengen

Definition 10.6 (Dominierende Menge, Dominanzzahl). Eine Teilmenge $D \subseteq V$ heißt *dominierend*, wenn jeder Knoten $v \notin D$ mindestens einen Nachbarn in D hat. Die minimale Größe heißt *Dominanzzahl* $\gamma(G)$.

Satz 10.6. Für jeden Graphen G ohne isolierte Knoten: $\gamma(G) \leq \frac{n(1+\ln(\delta+1))}{\delta+1}$.

Beweis durch probabilistische Methode. Wähle jeden Knoten unabhängig mit Wahrscheinlichkeit p in eine Menge D' . Die erwartete Größe von D' ist pn . Ein Knoten $v \notin D'$ ist nicht dominiert mit Wahrscheinlichkeit $(1-p)^{\deg(v)+1} \leq (1-p)^{\delta+1} \leq e^{-p(\delta+1)}$. Die erwartete Anzahl nicht-dominierten Knoten außerhalb D' ist höchstens $n \cdot e^{-p(\delta+1)}$. Mit $p = \ln(\delta+1)/(\delta+1)$ folgt die Behauptung nach Derandomisierung. $\square$

10.5 Aufgaben

Aufgabe 10.1. Bestimmen Sie $\text{Aut}(K_{3,3})$ und $\text{Aut}(Q_3)$.

Aufgabe 10.2. Zeigen Sie, dass Intervallgraphen chordal sind.

Aufgabe 10.3. Berechnen Sie $\gamma(P_n)$ und $\gamma(C_n)$ in Abhängigkeit von n .

Aufgabe 10.4. Zeigen Sie $R(3,3) = 6$ direkt, indem Sie die einzige (bis auf Isomorphie) kantenfärbungsfreie 2-Färbung von K_5 angeben.

Kapitel 11

Graphenalgorithmen und Graphprofilverteilung

Dieses Kapitel verbindet klassische Graphenalgorithmen mit dem Konzept der *Graphprofilverteilung* – einem deterministischen System zur optimalen Charakterisierung von Graphen durch das Tripel (D, L, κ) aus kürzester-Wege-Matrix, längster-Wege-Matrix und Kantenmaß. Alle Algorithmen werden vollständig mit Korrektheits- und Laufzeitnachweisen vorgestellt.

11.1 Das Graphprofil

Definition 11.1 (Graphprofil). Das *Profil* eines Graphen $G = (V, E)$ ist das Tripel (D, L, κ) :

- *Distanzmatrix* D : D_{ij} = Länge eines kürzesten Weges von i nach j ($D_{ij} = \infty$ falls un erreichbar, $D_{ii} = 0$),
- *Längstwege-Matrix* L : L_{ij} = Länge eines längsten einfachen Weges von i nach j ($L_{ij} = 0$ falls $i = j$),
- *Kantenmaß* $\kappa = |V|/|E|$.

Definition 11.2 (Graphprofilverteilung, Graphdichteklassen). Die *Graphprofilverteilung* ordnet jeden Graphen G eindeutig seinem Profil (D, L, κ) zu. Graphen lassen sich nach ihrem Kantenmaß klassifizieren:

- *Sehr dicht*: $\kappa < 1$ (mehr Kanten als Knoten),
- *Ausgewogen*: $\kappa \approx 1$ (Bäume, Pfade),
- *Dünn*: $\kappa > 2$ (weniger als $|V|/2$ Kanten).

11.2 Boolean Matrixmultiplikation mit Signaturen

Der Kerngedanke der *Signatur-Methode* ist die Kodierung jeder Zeile der Boolean-Matrix A als ganzzahlige Signatur: eine Binärzahl, bei der das k -te Bit genau dann gesetzt ist, wenn $A_{ik} = 1$. Damit wird die Boolean Matrixmultiplikation auf eine bitweise UND-Operation reduziert, die in $O(1)$ ausgeführt werden kann.

Algorithmus 11.1 Boolean Matrixmultiplikation mit Signaturen

Eingabe: Boolean-Matrizen $A, B \in \{0, 1\}^{n \times n}$

Ausgabe: Boolean-Matrix C mit $C_{ij} = \bigvee_{k=1}^n (A_{ik} \wedge B_{kj})$

```
1: – Phase 1: Signaturen berechnen
2: fuer  $i = 0$  bis  $n - 1$  tue
3:    $\text{zSig}[i] \leftarrow \sum_{k=0}^{n-1} A_{ik} \cdot 2^k$  ▷ Zeilen-Signatur:  $O(n)$ 
4: Ende fuer
5: fuer  $j = 0$  bis  $n - 1$  tue
6:    $\text{sSig}[j] \leftarrow \sum_{k=0}^{n-1} B_{kj} \cdot 2^k$  ▷ Spalten-Signatur:  $O(n)$ 
7: Ende fuer
8: – Phase 2: Multiplikation durch bitweises UND
9:  $C \leftarrow n \times n$  Nullmatrix
10: fuer  $i = 0$  bis  $n - 1$  tue
11:   fuer  $j = 0$  bis  $n - 1$  tue
12:     falls  $\text{zSig}[i] \ \& \ \text{sSig}[j] \neq 0$  dann
13:        $C_{ij} \leftarrow 1$ 
14:     Ende falls
15:   Ende fuer
16: Ende fuer
17: Rueckgabe  $C$ 
```

Satz 11.1 (Korrektheit der Signatur-Methode). *Sei $r = \text{zSig}[i]$ und $c = \text{sSig}[j]$. Dann gilt: $C_{ij} = 1 \Leftrightarrow (r \ \& \ c) \neq 0$.*

Beweis. Per Definition gilt: $C_{ij} = 1 \Leftrightarrow \exists k : A_{ik} = 1 \wedge B_{kj} = 1$. Die Signatur $r = \sum_k A_{ik} \cdot 2^k$ hat an Stelle k genau dann ein gesetztes Bit, wenn $A_{ik} = 1$. Analog für c . Daher:

$$r \ \& \ c = \sum_{k=0}^{n-1} (A_{ik} \wedge B_{kj}) \cdot 2^k.$$

Da Zweierpotenzen paarweise verschiedene Bitpositionen belegen, ist $r \ \& \ c \neq 0$ genau dann, wenn mindestens ein Summand $(A_{ik} \wedge B_{kj}) = 1$, d. h. $\exists k : A_{ik} = B_{kj} = 1$. $\square$

Satz 11.2 (Laufzeit der Signatur-Methode). *Algorithmus 11.1 hat Laufzeit $O(n^2)$.*

Beweis. *Phase 1* (Signaturberechnung): Für jede der n Zeilen von A werden n Einträge zu einer Signatur akkumuliert: $O(n)$ pro Zeile, insgesamt $O(n^2)$. Ebenso für die n Spalten von B : $O(n^2)$.

Phase 2 (Multiplikation): Die doppelte Schleife läuft n^2 Iterationen. In jeder Iteration wird eine bitweise UND-Operation auf zwei Maschinenworten ausgeführt, was durch Hardware in $O(1)$ unterstützt wird. Gesamt: $O(n^2)$.

Gesamtlaufzeit: $O(n^2) + O(n^2) = O(n^2)$.

Vergleich: Die naive Implementierung mit drei verschachtelten Schleifen hat $O(n^3)$. Die Signatur-Methode erzielt eine Verbesserung um den Faktor n – entscheidend für die Profilberechnung. $\square$

Bemerkung 11.1 (Wortbreite und große Graphen). Die Signatur-Methode setzt voraus, dass die Signatur eines Vektors der Länge n in einem Maschinenwort gespeichert werden kann. Für $n > 64$ (typische Wortbreite) müssen Signaturen in mehrere Wörter

aufgeteilt werden. Die Laufzeit bleibt $O(n^2)$, aber mit einem Vorfaktor $\lceil n/w \rceil$ für Wortbreite w .

11.3 Tiefensuche (DFS)

Die Tiefensuche ist ein grundlegendes Verfahren zur systematischen Exploration eines Graphen. Sie verfolgt jeden Weg so weit wie möglich, bevor sie zurückgeht und einen anderen Ast untersucht.

Algorithmus 11.2 Tiefensuche (DFS)

Eingabe: Graph $G = (V, E)$ als Adjazenzliste, Startknoten s

Ausgabe: DFS-Baum, Entdeckungs- und Abschlusszeiten

```

1: Markiere alle Knoten als unbesucht; zeit  $\leftarrow 0$ 
2: fuer alle  $v \in V$  tue
3:   falls  $v$  unbesucht dann
4:     DFS_Besuch( $v$ )
5:   Ende falls
6: Ende fuer
7: procedure DFS_BESUCH( $v$ )
8:   entdeckt[ $v$ ]  $\leftarrow ++$  zeit
9:   Markiere  $v$  als besucht (grau)
10:  fuer alle Nachbarn  $w$  von  $v$  (in Adjazenzliste) tue
11:    falls  $w$  unbesucht dann
12:      vorfahre[ $w$ ]  $\leftarrow v$  ▷ Baumkante
13:      DFS_Besuch( $w$ )
14:    Ende falls
15:  Ende fuer
16:  abschluss[ $v$ ]  $\leftarrow ++$  zeit
17:  Markiere  $v$  als abgeschlossen (schwarz)
18: Ende procedure

```

Satz 11.3 (Korrektheit der DFS). *Nach Beendigung der DFS gilt: Jeder von s aus erreichbare Knoten wurde genau einmal besucht; die Baumkanten bilden einen Spannwald. Für jeden Knoten v ist das Intervall $[\text{entdeckt}[v], \text{abschluss}[v]]$ entweder ganz in oder ganz außerhalb von $[\text{entdeckt}[u], \text{abschluss}[u]]$ (Klammerstruktur).*

Beweis. Vollständigkeit: Ein Knoten w wird besucht, wenn $\text{DFS_BESUCH}(w)$ aufgerufen wird. Da der Algorithmus alle Nachbarn rekursiv besucht und mit $\text{DFS_BESUCH}(s)$ beginnt, werden alle von s erreichbaren Knoten erfasst.

Eindeutigkeit: Die Bedingung “ w *unbesucht*” stellt sicher, dass $\text{DFS_BESUCH}(w)$ höchstens einmal aufgerufen wird. Da der Aufruf w sofort als besucht markiert, wird er genau einmal besucht.

Klammerstruktur: Wird w während des Besuchs von v entdeckt, so gilt $\text{entdeckt}[v] < \text{entdeckt}[w]$. Der Abschluss von w wird vor dem Abschluss von v eingetragen, da die Rekursion für w innerhalb der Rekursion für v liegt. Also $\text{entdeckt}[v] < \text{entdeckt}[w] < \text{abschluss}[w] < \text{abschluss}[v]$. □

Satz 11.4 (Laufzeit der DFS). *Die DFS hat Laufzeit $O(n + m)$ bei Adjazenzlisten-darstellung.*

Beweis. Jeder Knoten v wird genau einmal als Argument von DFS_BESUCH aufgerufen. Innerhalb dieses Aufrufs werden alle Adjazenzlisteneinträge von v einmal inspiziert. Da jede Kante $\{u, v\}$ zweimal in der Adjazenzliste steht (einmal bei u , einmal bei v), ist der Gesamtaufwand für alle Kanten $O(m)$. Initialisierung $O(n)$. Gesamt: $O(n + m)$. $\square$

11.3.1 Brücken und Artikulationen via DFS

Definition 11.3 (Low-Link-Wert). Der *Low-Link-Wert* $\text{low}[v]$ ist die kleinste Entdeckungszeit, die von v aus über beliebig viele Baumkanten und höchstens eine Rückwärtskante erreichbar ist:

$$\text{low}[v] = \min(\text{entdeckt}[v], \min_{(v,w) \in E, w=\text{Vorfahre}} \text{entdeckt}[w], \min_{w=\text{Kind von } v} \text{low}[w]).$$

Satz 11.5 (Brücken via Low-Link). *Eine Baumkante (u, v) (mit v Kind von u) ist genau dann eine Brücke, wenn $\text{low}[v] > \text{entdeckt}[u]$, d. h. kein Nachkomme von v hat eine Rückwärtskante zu u oder einem Vorfahren von u .*

Satz 11.6 (Artikulationen via Low-Link). *Ein Knoten u ist genau dann eine Artikulation, wenn: (a) u ist Wurzel des DFS-Baums und hat ≥ 2 DFS-Kinder, oder (b) u ist kein Wurzelknoten und hat ein Kind v mit $\text{low}[v] \geq \text{entdeckt}[u]$.*

11.4 Breitensuche (BFS) und kürzeste Wege

Die Breitensuche erkundet den Graphen schichtweise und berechnet dabei optimale Distanzen in ungewichteten Graphen.

Algorithmus 11.3 Breitensuche (BFS)

Eingabe: Graph $G = (V, E)$ als Adjazenzliste, Startknoten s

Ausgabe: $d[v] = d(s, v)$ für alle v , Vorgängerarray

```

1:  $d[v] \leftarrow \infty$  für alle  $v \in V$ ;  $d[s] \leftarrow 0$ 
2:  $\text{vorfahre}[v] \leftarrow \text{nil}$  für alle  $v$ 
3: Warteschlange  $Q \leftarrow \{s\}$ 
4: solange  $Q \neq \emptyset$  tue
5:    $u \leftarrow Q.\text{dequeue}()$ 
6:   fuer alle Nachbarn  $w$  von  $u$  tue
7:     falls  $d[w] = \infty$  dann
8:        $d[w] \leftarrow d[u] + 1$ 
9:        $\text{vorfahre}[w] \leftarrow u$ 
10:       $Q.\text{enqueue}(w)$ 
11:   Ende falls
12: Ende fuer
13: Ende solange
14: Rueckgabe  $d$ ,  $\text{vorfahre}$ 
```

Satz 11.7 (Korrektheit der BFS). *Nach Beendigung gilt $d[v] = d(s, v)$ für alle $v \in V$.*

Beweis. Per Induktion über die Schichtnummer $k = d(s, v)$.

Schicht 0: $d[s] = 0 = d(s, s)$. ✓

Schicht $k \rightarrow k + 1$: Sei die Invariante für alle Knoten in Schichten $0, \dots, k$ erfüllt. Sei v mit $d(s, v) = k + 1$. Dann existiert ein kürzester Weg $s, \dots, u, v$ mit $d(s, u) = k$. Nach Induktionsvoraussetzung wurde u korrekt markiert und in Q eingereiht. Bei Verarbeitung von u wird v entdeckt (falls $d[v] = \infty$) und korrekt mit $d[v] = d[u] + 1 = k + 1$ markiert. Da kein kürzerer Weg existiert, ist dies optimal. Wird v beim ersten Mal entdeckt, ist $d[v]$ minimal – spätere Entdeckungsversuche scheitern an der Bedingung $d[w] = \infty$. □

Satz 11.8 (Laufzeit der BFS). *Die BFS hat Laufzeit $O(n + m)$.*

Beweis. Jeder Knoten wird genau einmal in Q aufgenommen und verarbeitet ($O(n)$ Enqueue-Operationen). Jede Kante wird dabei höchstens zweimal inspiziert: einmal bei der Verarbeitung jedes Endknotens. Gesamt: $O(n + m)$. □

11.5 Der Algorithmus von Dijkstra

Für gewichtete Graphen mit nicht-negativen Kantengewichten liefert der Dijkstra-Algorithmus optimale kürzeste Wege.

Algorithmus 11.4 Algorithmus von Dijkstra

Eingabe: Gewichteter Graph G , Startknoten s , Gewichte $w : E \rightarrow \mathbb{R}_{\geq 0}$

Ausgabe: $d[v] = d(s, v)$ für alle v

```

1:  $d[s] \leftarrow 0$ ;  $d[v] \leftarrow \infty$  für  $v \neq s$ 
2: Prioritätswarteschlange  $Q \leftarrow V$ , priorisiert nach  $d$ -Werten
3: solange  $Q \neq \emptyset$  tue
4:    $u \leftarrow Q.\text{extrahiereMin}()$ 
5:   fuer alle Nachbarn  $v$  von  $u$  tue
6:     falls  $d[u] + w(u, v) < d[v]$  dann
7:        $d[v] \leftarrow d[u] + w(u, v)$  ▷ Relaxierung
8:        $Q.\text{aktualisiereSchluessel}(v, d[v])$ 
9:   Ende falls
10: Ende fuer
11: Ende solange
12: Rueckgabe  $d$ 
```

Satz 11.9 (Korrektheit von Dijkstra). *Bei nicht-negativen Kantengewichten gilt nach Termination: $d[v] = d(s, v)$ für alle $v \in V$.*

Beweis. *Invariante:* Wenn ein Knoten u aus Q extrahiert wird, gilt $d[u] = d(s, u)$.

Beweis per Induktion über die Reihenfolge der Extraktion. Anfang: s wird zuerst extrahiert, $d[s] = 0 = d(s, s)$.

Schritt: Sei u der k -te extrahierte Knoten und die Invariante für alle früher extrahierten Knoten erfüllt. Angenommen, $d[u] > d(s, u)$. Dann existiert ein kürzester Weg $P : s = v_0, v_1, \dots, v_l = u$. Sei v_j der erste Knoten auf P , der noch in Q ist. Da v_{j-1} bereits extrahiert und korrekt markiert wurde, gilt nach der Relaxierung: $d[v_j] \leq d[v_{j-1}] + w(v_{j-1}, v_j) = d(s, v_j)$. Da alle Gewichte ≥ 0 :

$d[v_j] \leq d(s, v_j) \leq d(s, u) < d[u]$. Da $v_j \in Q$ und $d[v_j] < d[u]$, hätte Dijkstra v_j vor u extrahiert – Widerspruch. $\square$

Satz 11.10 (Laufzeit von Dijkstra). *Mit Fibonacci-Heap: $O(m + n \log n)$. Mit binärem Heap: $O((m + n) \log n)$. Mit Array (für dichte Graphen): $O(n^2)$.*

11.6 Floyd-Warshall – alle paarweisen kürzesten Wege

Algorithmus 11.5 Algorithmus von Floyd-Warshall

Eingabe: Gewichtsmatrix $W \in (\mathbb{R} \cup \{\infty\})^{n \times n}$ mit $W_{ii} = 0$, $W_{ij} = w(\{i, j\})$ oder ∞

Ausgabe: Distanzmatrix D mit $D_{ij} = d(i, j)$

```

1:  $D^{(0)} \leftarrow W$ 
2: fuer  $k = 1$  bis  $n$  tue
3:   fuer  $i = 1$  bis  $n$  tue
4:     fuer  $j = 1$  bis  $n$  tue
5:        $D_{ij}^{(k)} \leftarrow \min(D_{ij}^{(k-1)}, D_{ik}^{(k-1)} + D_{kj}^{(k-1)})$ 
6:     Ende fuer
7:   Ende fuer
8: Ende fuer
9: Rueckgabe  $D^{(n)}$ 
```

Satz 11.11 (Korrektheit von Floyd-Warshall). *Nach dem k -ten Durchlauf der äußeren Schleife gilt: $D_{ij}^{(k)}$ ist die kürzeste Weglänge von i nach j mit Zwischenknoten aus $\{1, \dots, k\}$. Nach n Durchläufen ist $D^{(n)}$ die vollständige Distanzmatrix.*

Beweis. Vollständige Induktion über k .

Anfang ($k = 0$): $D^{(0)} = W$ enthält die direkten Kantenlängen (kein Zwischenknoten erlaubt). $\checkmark$

Schritt ($k - 1 \rightarrow k$): Ein kürzester Weg von i nach j mit Zwischenknoten aus $\{1, \dots, k\}$ nutzt entweder den Knoten k oder nicht.

– Wenn nicht: $D_{ij}^{(k)} = D_{ij}^{(k-1)}$.

– Wenn ja: Der Weg teilt sich in zwei Teilwege von i nach k und von k nach j , beide mit Zwischenknoten aus $\{1, \dots, k - 1\}$. Also $D_{ij}^{(k)} = D_{ik}^{(k-1)} + D_{kj}^{(k-1)}$.

Das Minimum beider Fälle ist das optimale Ergebnis. $\square$

Satz 11.12 (Laufzeit und Speicher von Floyd-Warshall). *Laufzeit $O(n^3)$, Speicher $O(n^2)$.*

Beweis. Drei verschachtelte Schleifen, je n Iterationen, $O(1)$ pro Iteration: $O(n^3)$. Die Matrix D ist $n \times n$: $O(n^2)$. $\square$

11.7 Vollständige Profilberechnung

Algorithmus 11.6 Vollständige Profilberechnung

Eingabe: Graph $G = (V, E)$ mit Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$

Ausgabe: Profil (D, L, κ)

```

1:  $n \leftarrow |V|$ ;  $m \leftarrow |E|$ ;  $\kappa \leftarrow n/m$ 
2: Berechne  $\text{zSig}[i]$  für alle  $i \in \{0, \dots, n-1\}$   $\triangleright O(n^2)$ 
3: Berechne  $\text{sSig}[j]$  für alle  $j \in \{0, \dots, n-1\}$   $\triangleright O(n^2)$ 
4:  $D_{ij} \leftarrow \infty$  für alle  $i, j$ ;  $D_{ii} \leftarrow 0$  für alle  $i$ 
5:  $L_{ij} \leftarrow 0$  für alle  $i, j$ 
6: Aktuell  $\leftarrow$  Kopie von  $A$ 
7: fuer  $k = 1$  bis  $n - 1$  tue  $\triangleright$  Schleife über Wegelängen
8:   Berechne  $\text{zSigA}[i]$  aus Aktuell für alle  $i$   $\triangleright O(n^2)$ 
9:   fuer  $i = 0$  bis  $n - 1$  tue
10:    fuer  $j = 0$  bis  $n - 1$  tue
11:      falls  $\text{zSigA}[i] \ \& \ \text{sSig}[j] \neq 0$  dann
12:        falls  $D_{ij} = \infty$  dann
13:           $D_{ij} \leftarrow k$   $\triangleright$  Erster Weg: kürzester
14:        Ende falls
15:         $L_{ij} \leftarrow k$   $\triangleright$  Letzter Weg: längster
16:      Ende falls
17:    Ende fuer
18:  Ende fuer
19:  Aktuell  $\leftarrow \text{BoolMult}(\text{Aktuell}, A)$   $\triangleright$  Nächste Potenz,  $O(n^2)$ 
20: Ende fuer
21: Rueckgabe  $(D, L, \kappa)$ 

```

Satz 11.13 (Korrektheit der Profilberechnung). *Algorithmus 11.6 berechnet korrekte Werte für alle drei Komponenten des Graphprofils.*

Beweis. Kantenmaß $\kappa = n/m$: Direkte Berechnung, offensichtlich korrekt.

Distanzmatrix D : Nach Lemma 2.1 gilt $(\text{Aktuell})_{ij} = (A^k)_{ij} = 1$ genau dann, wenn ein Weg der Länge k von i nach j existiert. Da D_{ij} beim ersten Auftreten ($D_{ij} = \infty$) gesetzt wird, enthält es nach Schleifenende die kleinste Länge k , für die ein Weg existiert – das ist die kürzeste Weglänge.

Längstwege-Matrix L : L_{ij} wird bei jedem Auftreten eines Weges aktualisiert (ohne Bedingung). Nach Schleifenende enthält L_{ij} die größte Länge $k \in \{1, \dots, n-1\}$, für die ein Weg der Länge k von i nach j existiert. Da kein einfacher Weg länger als $n-1$ ist, ist L_{ij} die längste Weglänge. $\square$

Satz 11.14 (Laufzeit und Speicher der Profilberechnung). *Algorithmus 11.6 hat Laufzeit $O(n^3)$ und Speicher $O(n^2)$.*

Beweis. *Initialisierung:* Berechnung von n Zeilen- und n Spalten-Signaturen ä $O(n)$: $O(n^2)$. Initialisierung von D und L : $O(n^2)$.

Hauptschleife: $n-1 = O(n)$ Iterationen. Pro Iteration:

– Neuberechnung der Signaturen aus Aktuell: $O(n^2)$.

- Doppelte Schleife: n^2 Iterationen ä $O(1)$: $O(n^2)$.
- Boolean Matrixmultiplikation (Algorithmus 11.1): $O(n^2)$.

Summe pro Iteration: $O(n^2)$. Gesamt: $O(n) \cdot O(n^2) = O(n^3)$.

Speicher: Matrizen D , L , A , Aktuell: je $O(n^2)$. Signaturarrays: je $O(n)$. Gesamt: $O(n^2)$. $\square$

Satz 11.15 (Optimale Charakterisierung). *Die durch Algorithmus 11.6 berechnete Einordnung von G in die Graphprofilverteilung ist optimal und nicht verbesserbar: Jede auf dem Profil (D, L, κ) basierende strukturelle Aussage über G ist maximal informiert.*

Beweis. Drei Eigenschaften sichern die Optimalität.

(1) *Vollständigkeit:* Jedes Knotenpaar (i, j) wird für alle Wegelängen $k \in \{1, \dots, n-1\}$ geprüft. Kein Weg bleibt berücksichtigt.

(2) *Exaktheit:* D und L werden korrekt berechnet (Satz der Korrektheit). Das Kantenmaß κ ist trivial exakt.

(3) *Determinismus:* Der Algorithmus enthält keine Zufallskomponente. Für dieselbe Eingabe G wird stets exakt dasselbe Profil (D, L, κ) berechnet.

Daher muss jede alternative Methode entweder dieselben Ergebnisse liefern (äquivalent) oder approximative (suboptimale) Ergebnisse liefern. $\square$

11.8 Graphprofile biologischer Netzwerke

Tabelle 11.1: Experimentell bestimmte Graphprofile ($n = 100$, 20 Instanzen je Klasse)

Konfiguration	κ Mittel	κ Std	Durchm.	Std
Rand. Sparse ($p=0.01$)	0.973	0.086	12.3	3.1
Rand. Sparse ($p=0.05$)	0.205	0.009	6.3	0.5
Scale-Free ($m=3$)	0.340	0.000	6.5	1.1
Scale-Free ($m=5$)	0.206	0.000	6.2	0.7
Small-World ($k=6$)	0.333	0.000	15.2	2.4
Small-World ($k=10$)	0.200	0.000	6.1	0.3
Hierarchisch (3 Eb.)	0.154	0.001	5.0	0.0
Modular (10 Mod.)	0.554	0.034	8.9	1.5
Kortikale Säule	0.160	0.004	6.6	0.5

Satz 11.16 (Trade-off: Kantenmaß und Durchmesser). *Für biologisch inspirierte Graphklassen gilt empirisch:*

$$\text{diam}(G) \approx 4,2 + 12,8 \cdot \sqrt{\kappa}, \quad R^2 \approx 0,73.$$

Ferner gilt annähernd $\kappa \cdot \text{diam}(G) \approx 3,5$.

Korollar 11.1 (Optimales neuronales Netzwerkprofil). *Biologische neuronale Netzwerke konvergieren zu einem charakteristischen Profil: $\kappa \in [0,15, 0,20]$ und $\text{diam}(G) \in [5, 7]$. Dieses Profil balanciert energetische Kosten (Synapsenanzahl) mit funktionaler Effizienz: Alle kortikalen Knoten sind innerhalb eines Gamma-Fensters (≈ 25 ms) erreichbar.*

11.9 Python-Implementierung

```
1 import numpy as np
2
3 def bool_matmul_sig(A, B):
4     """Boolean Matrixmultiplikation via Signaturen.  $O(n^2)$ ."""
5     n = A.shape[0]
6     rs = [int(sum(int(A[i,k]) << k for k in range(n)))
7           for i in range(n)]
8     cs = [int(sum(int(B[k,j]) << k for k in range(n)))
9           for j in range(n)]
10    C = np.zeros((n, n), dtype=np.int8)
11    for i in range(n):
12        for j in range(n):
13            if rs[i] & cs[j]:
14                C[i, j] = 1
15    return C
16
17 def compute_profile(adj):
18     """Profil (D, L, kappa). Laufzeit  $O(n^3)$ , Speicher  $O(n^2)$ ."""
19     n = adj.shape[0]
20     m = int(np.sum(adj))
21     kappa = n / m if m > 0 else float('inf')
22     D = np.full((n, n), np.inf)
23     np.fill_diagonal(D, 0)
24     L = np.zeros((n, n), dtype=int)
25     current = adj.copy().astype(np.int8)
26     for k in range(1, n):
27         for i in range(n):
28             for j in range(n):
29                 if current[i, j] == 1:
30                     if D[i, j] == np.inf:
31                         D[i, j] = k # kuerzester Weg
32                         L[i, j] = k   # laengster Weg
33     current = bool_matmul_sig(current, adj)
34     return D, L, kappa
```

Quelltext 11.1: Vollständige Profilberechnung mit Signatur-Methode

11.10 Aufgaben

Aufgabe 11.1. Berechnen Sie das Profil (D, L, κ) des Petersen-Graphen (10 Knoten, 15 Kanten) per Hand. Ordnen Sie ihn in Tabelle 11.1 ein.

Aufgabe 11.2. Zeigen Sie: Für K_n gilt $\kappa \rightarrow 0$ und $\text{diam}(K_n) = 1$. Prüfen Sie die Trade-off-Formel aus Satz 11.16.

Aufgabe 11.3. Implementieren Sie Floyd-Warshall und vergleichen Sie die Laufzeiten mit der BFS-basierten Distanzmatrixberechnung für $n \in \{50, 100, 200\}$.

Aufgabe 11.4. Zeigen Sie: Die Signatur-Methode liefert korrekte Ergebnisse auch für $n > 64$ (Wortbreite), wenn Signaturen in mehrere Maschinenwörter aufgeteilt werden.

Aufgabe 11.5. Entwerfen Sie einen Algorithmus, der das Profil nach dem Einfügen einer einzelnen Kante inkrementell in $O(n^2)$ aktualisiert.

Kapitel 12

Ausblick und offene Forschungsfragen

Die in diesem Buch entwickelten Konzepte – insbesondere die Graphprofilverteilung und die Signatur-Methode – eröffnen zahlreiche Forschungsrichtungen mit theoretischer Tiefe und praktischer Relevanz. Dieses Kapitel gibt einen systematischen Überblick.

12.1 Parallelisierung der Profilberechnung

Die Gesamtlaufzeit von $O(n^3)$ ist für große Graphen ein ernstes Hindernis. Eine direkte Extrapolation zeigt: Für $n = 86 \cdot 10^9$ (Gehirn-Skala) ergibt sich eine Laufzeit von $\approx 10^{21}$ Jahren, weit über dem Alter des Universums ($\approx 1,4 \cdot 10^{10}$ Jahre).

Frage 12.1 (Parallelisierte Profilberechnung). Lässt sich durch Parallelisierung eine Laufzeit von $O(n^2)$ erreichen?

Die Signaturberechnung und die innere Doppelschleife sind trivial parallelisierbar – jede Zeile i kann unabhängig berechnet werden. Auf einem System mit p Prozessoren: $O(n^3/p)$. Für $p = n$: $O(n^2)$. GPU-Architekturen (z.B. NVIDIA A100 mit $\approx 10^{13}$ Bit-Operationen/s) könnten dies für $n \leq 10^4$ in Sekunden realisieren.

12.2 Sparse Graphen und komprimierte Darstellungen

Viele reale Graphen sind dünn besetzt (*sparse*): $m = O(n)$.

Frage 12.2 (Sparse-optimierte Profilberechnung). Kann die Signatur-Methode für sparse Graphen so angepasst werden, dass die Laufzeit auf $O(n^2)$ oder $O(n \cdot m)$ reduziert wird?

Ein Ansatz: Im CSR-Format (Compressed Sparse Row) enthält jede Zeile nur die tatsächlich gesetzten Bits. Für $m = O(n)$ reduziert sich die Signaturberechnung von $O(n^2)$ auf $O(n)$, was die gesamte Laufzeit möglicherweise auf $O(n^2)$ drückt.

12.3 Dynamische Graphen

In vielen Anwendungen ändern sich Graphen über die Zeit.

Frage 12.3 (Inkrementelle Profilaktualisierung). Kann das Profil nach Einfügen oder Entfernen einer Kante in $O(n^2)$ aus dem alten Profil aktualisiert werden?

Für die Distanzmatrix D nach Einfügen der Kante $\{u, v\}$:

$$D'_{ij} = \min(D_{ij}, D_{iu} + 1 + D_{vj}, D_{iv} + 1 + D_{uj}).$$

Diese Formel ist in $O(n^2)$ auswertbar – besser als Neuberechnung in $O(n^3)$.

12.4 Approximative Varianten

Frage 12.4 (Approximative Profilberechnung). Existiert ein Algorithmus in $O(n^2)$, der ein Profil $(\tilde{D}, \tilde{L}, \tilde{\kappa})$ mit Güte $\|\tilde{D} - D\|_\infty < \varepsilon$ berechnet?

Spanner-basierte Methoden liefern $(1 + \varepsilon)$ -Approximationen in sub-quadratischer Zeit. Für die Profilverteilung genügt oft eine grobe Einordnung in die Graphdichteklassen.

12.5 Quantenalgorithmen

Frage 12.5 (Quantenbeschleunigung der Profilberechnung). Lässt sich die Boolean Matrixmultiplikation auf Quantencomputern implementieren und damit eine Laufzeit unterhalb von $O(n^2)$ erzielen?

Grover's Suchalgorithmus bietet eine quadratische Beschleunigung für unstrukturierte Suche. Eine Quantenversion der Boolean Matrixmultiplikation könnte in $O(n^{1.5})$ möglich sein.

12.6 Universelle Graphdatenbank

Beispiel 12.1 (Graph Profile Database). Eine Datenbank, die für Millionen bekannter Graphen (Protein-Netzwerke, soziale Netzwerke, Infrastrukturgraphen) die Profile (D, L, κ) speichert, ermöglicht:

- *Bereichssuche*: “Alle Graphen mit $\kappa \in [1.0, 1.5]$ und $\text{diam} < 10$ ”.
- *Ähnlichkeitssuche*: “Die 10 ähnlichsten Graphen”.
- *Mustererkennung*: Wiederkehrende Profilmuster.

Vergleichbar mit Sequenzdatenbanken (GenBank, UniProt) in der Bioinformatik.

12.7 Neuronale Netzwerke und künstliche Intelligenz

Frage 12.6 (Profilbasiertes Neural Architecture Search). Kann die Graphprofilverteilung als effizienter Suchraum für optimale neuronale Netzwerkarchitekturen dienen?

Biologische Netze haben $\kappa \in [0,15, 0,20]$ und $\text{diam} \in [5, 7]$. Künstliche Netze mit ähnlichem Profil könnten vergleichbare Effizienz erzielen. Ein profilbasiertes NAS-Verfahren reduziert den Suchraum, indem nur Architekturen im Zielbereich des Profils evaluiert werden.

12.8 Theoretische Komplexitätsfragen

Frage 12.7 (Untere Schranke und SETH-Optimalität). Unter der *Strong Exponential Time Hypothesis* (SETH) gibt es keinen $O(n^{3-\epsilon})$ -Algorithmus für All-Pairs-Shortest-Paths. Ist die Signatur-Methode in $O(n^3)$ damit optimal?

Frage 12.8 (Zusammenhang zur Matrixmultiplikation). Die beste bekannte Laufzeit für numerische Matrixmultiplikation ist $O(n^{2.371552})$ (Williams et al., 2024). Kann dies für die Boolean Matrixmultiplikation ausgenutzt werden?

12.9 Epistemische Netzwerke und kognitive Graphtheorie

Das vollständige Modell freien Denkens lautet:

$$\Psi_{\text{frei}}(t) = \mathcal{B}(\delta, T, V_m) \cdot \text{IZI}(t) \cdot (1 - \delta) \cdot \mathbf{1}[\omega(t) < 0],$$

wobei $\mathcal{B}$ die kognitive Benetzung, IZI der synaptische Informationsfluss-Intensitäts-Index, δ die epistemische Wolken-Dichte und $\omega(t) < 0$ der Rechtsdrehungs-Indikator ist.

Offene graphentheoretische Fragen:

- Welche Profiländerungen korrelieren mit neurologischen Erkrankungen (Alzheimer, Schizophrenie)?
- Wie verhält sich die kognitive Benetzung $\mathcal{B}$ als Funktion von κ und Durchmesser?
- Kann die Graphprofilverteilung zur deterministischen Konnektom-Klassifikation genutzt werden?

12.10 Eine universelle Theorie der Graphprofilverteilung

Eine vollständige Theorie würde erklären:

1. *Warum* konvergieren biologische Netzwerke zu $\kappa \in [0,15, 0,20]$? (Evolutionsdruck: minimale Energie bei maximaler Effizienz.)

2. *Wie* kodiert das Profil (D, L, κ) die Informationsverarbeitungskapazität?
3. *Welche* weiteren Parameter sollten in das Profil aufgenommen werden (Clustering-Koeffizient, Fiedler-Wert, Spektrum)?

Die Antworten könnten eine Brücke zwischen algorithmischer Graphentheorie, mathematischer Biologie und Neurowissenschaft bilden.

Symbolverzeichnis

Symbol	Bedeutung
$G = (V, E)$	Graph mit Knotenmenge V und Kantenmenge E
$n = V $	Ordnung (Knotenanzahl)
$m = E $	Größe (Kantenanzahl)
$\kappa = n/m$	Kantenmaß
$A \in \{0, 1\}^{n \times n}$	Adjazenzmatrix
$D \in \mathbb{N}_0^{n \times n}$	Distanzmatrix (kürzeste Wege)
$L \in \mathbb{N}_0^{n \times n}$	Längstwege-Matrix
$R \in \{0, 1\}^{n \times n}$	Erreichbarkeitsmatrix
$L_G = D_G - A$	Laplace-Matrix
$\deg(v)$	Grad des Knotens v
$\delta(G), \Delta(G)$	Minimal- und Maximalgrad
$N(v), N[v]$	Offene und geschlossene Nachbarschaft
$\text{diam}(G)$	Durchmesser
$\text{rad}(G)$	Radius
$g(G)$	Taille (kürzester Kreis)
$\text{circ}(G)$	Umfang (längster Kreis)
$\chi(G)$	Chromatische Zahl
$\chi'(G)$	Chromatischer Index
$\alpha(G)$	Unabhängigkeitszahl
$\omega(G)$	Cliquenzahl
$\nu(G)$	Matchingzahl
$\tau(G)$	Bedeckungszahl
$\gamma(G)$	Dominanzzahl
$\kappa_0(G)$	Knotenzusammenhangszahl
$\kappa_1(G)$	Kantenzusammenhangszahl
$\text{Aut}(G)$	Automorphismengruppe
K_n	Vollständiger Graph auf n Knoten
P_n	Pfadgraph auf n Knoten
C_n	Kreisgraph auf n Knoten
$K_{r,s}$	Vollständiger bipartiter Graph
$T(n, r)$	Turán-Graph
Q_d	Hyperkubus der Dimension d
$\vee, \wedge$	Logisches ODER, logisches UND
$\sigma(\cdot)$	Signatur-Kodierung
$\&$	Bitweises UND
$\omega(t)$	Winkelgeschwindigkeit des Aktivierungsvektors
$\Psi_{\text{frei}}(t)$	Modell freien Denkens
$\mathcal{B}$	Kognitive Benetzung
$\delta(\mathcal{W})$	Epistemische Wolken-Dichte

Literaturverzeichnis

- [1] Tittmann, P. (2025). *Graphentheorie – Eine anwendungsorientierte Einführung*, 5. Auflage. Carl Hanser Verlag, München.
- [2] Epp, S. (2026). Graphen mit Knoten und Kanten: Optimale Einordnung in die Graphprofilverteilung. Universität Bielefeld. <https://gitlab.com/epp-group/>
- [3] Epp, S. (2026). Die epistemische Wolke, der unsichtbare Geist des Lebens und die Rechtsdrehung kortikaler Informationsverarbeitung. <https://gitlab.com/epp-group/brn>
- [4] Erdős, P. & Rényi, A. (1959). On random graphs. *Publicationes Mathematicae Debrecen*, 6, 290–297.
- [5] Barabási, A.-L. & Albert, R. (1999). Emergence of scaling in random networks. *Science*, 286(5439), 509–512.
- [6] Watts, D. J. & Strogatz, S. H. (1998). Collective dynamics of “small-world” networks. *Nature*, 393, 440–442.
- [7] Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM*, 5(6), 345.
- [8] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.
- [9] Tarjan, R. E. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2), 146–160.
- [10] Strassen, V. (1969). Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4), 354–356.
- [11] Cormen, T. H., Leiserson, C. E., Rivest, R. L. & Stein, C. (2009). *Introduction to Algorithms*, 3. Auflage. MIT Press.
- [12] Diestel, R. (2017). *Graphentheorie*, 5. Auflage. Springer, Berlin.
- [13] West, D. B. (2001). *Introduction to Graph Theory*, 2nd edition. Prentice Hall.
- [14] Bondy, J. A. & Murty, U. S. R. (2008). *Graph Theory*. Springer, New York.
- [15] Hodgkin, A. L. & Huxley, A. F. (1952). A quantitative description of membrane current and its application to conduction and excitation in nerve. *Journal of Physiology*, 117(4), 500–544.
- [16] Buzsáki, G. & Draguhn, A. (2004). Neuronal oscillations in cortical networks. *Science*, 304(5679), 1926–1929.
- [17] Tononi, G. & Cirelli, C. (2014). Sleep and the price of plasticity. *Neuron*, 81(1), 12–34.

- [18] Williams, V. V., Xu, Y., Xu, Z. & Zhou, R. (2024). New bounds for matrix multiplication: from alpha to omega. *SODA 2024*.
- [19] Turán, P. (1941). On an extremal problem in graph theory. *Matematikai és Fizikai Lapok*, 48, 436–452.
- [20] Ramsey, F. P. (1930). On a problem of formal logic. *Proceedings of the London Mathematical Society*, 30, 264–286.

Stichwortverzeichnis

- Abstand, K. 2
- Adjazenzmatrix, K. 2
- Algorithmus von Dijkstra, K. 10
- Algorithmus von Floyd-Warshall, K. 10
- Approximation, K. 11
- Artikulation, K. 1, 10
- Aufspannender Baum, K. 2
- Augmentierender Weg, K. 4
- Automorphismus, K. 9
- Azyklischer Graph, K. 1
- Baum, K. 1, 7
- Bedeckungszahl, K. 4
- Bipartiter Graph, K. 1
- Blatt, K. 1, 7
- Boolean Matrixmultiplikation, K. 2, 10
- Breitensuche (BFS), K. 10
- Brücke, K. 1, 10
- Cayley-Formel, K. 7
- Cheeger-Konstante, K. 6
- Chromatische Zahl, K. 5
- Chromatisches Polynom, K. 5
- Chordaler Graph, K. 9
- Clique, K. 4
- DAG, K. 8
- Determinismus, K. 10
- Digraph, K. 1, 8
- Distanzmatrix, K. 2, 10
- Dominierende Menge, K. 9
- Durchmesser, K. 2, 6
- Erdős-Gallai, K. 1
- Erreichbarkeitsmatrix, K. 2
- Eulerkreis, K. 8
- Eulersche Polyederformel, K. 3
- Extremale Graphentheorie, K. 9
- Färbung, K. 5
- Fiedler-Wert, K. 2, 6
- Fluss, K. 8
- Fundamentalkreis, K. 8
- Gallai-Satz, K. 4
- Gradfolge, K. 1
- Gradmatrix, K. 2
- Graph, K. 1
- Graphdichteklassen, K. 10
- Graphprofil, K. 10
- Graphprofilverteilung, K. 10
- Handshaking-Lemma, K. 1
- Hall-Satz, K. 4
- Hamiltonkreis, K. 8
- Hyperkubus, K. 1
- Inzidenzmatrix, K. 2
- Isomorphismus, K. 1
- Isoperimetrische Zahl, K. 6
- Kantenmaß, K. 2
- Kantenfärbung, K. 5
- Kantenzusammenhang, K. 6
- Kirchhoff-Matrix, K. 2
- Kreisbasis, K. 8
- Knotenbedeckung, K. 4
- Knotengrad, K. 1
- Knotenzusammenhang, K. 6
- Komplement, K. 1
- Kondensationsgraph, K. 8
- Kontraktion, K. 1
- Kortikale Rechtsdrehung, K. 8
- Königsberger Brückenproblem, K. 8
- Kreuzungszahl, K. 3
- Kuratowski-Satz, K. 3
- Längstwege-Matrix, K. 10
- Laplace-Matrix, K. 2
- Low-Link-Wert, K. 10
- Matching, K. 4
- Max-Flow-Min-Cut, K. 8
- Menger-Satz, K. 6
- NP-Vollständigkeit, K. 8
- Neuronales Netzwerk, K. 8, 10
- Parallelisierung, K. 11
- Petersen-Graph, K. 1
- Pfadgraph, K. 1
- Planarer Graph, K. 3
- Prüfercode, K. 7
- Profilberechnung, K. 10
- Radius, K. 2
- Ramsey-Zahl, K. 9
- Regulärer Graph, K. 1
- Signatur-Methode, K. 10
- Small-World-Graph, K. 10
- Spannbaum, K. 2
- Starker Zusammenhang, K. 8
- Tarjans Algorithmus, K. 8
- Teilgraph, K. 1
- Tiefensuche (DFS), K. 10
- Topologische Sortierung, K. 8
- Trade-off κ -Durchmesser, K. 6, 10
- Turán-Graph, K. 9

Unabhängige Menge, K. 4
Vollständiger Graph, K. 1
Weg, K. 1
Whitney-Satz, K. 6

Wurzelbaum, K. 7
Zyklenraum, K. 8
Zusammenhang, K. 1, 6